



西安电子科技大学
XIDIAN UNIVERSITY

《操作系统》

第四章 文件系统



人工智能学院

目录

- 一、基本概念
- 二、文件的结构
- 三、文件的目录
- 四、文件系统的实现
- 五、文件共享
- 六、文件系统的性能问题
- 七、文件系统的可靠性
- 八、文件系统的安全性

考纲内容

（一）文件

文件的基本概念；文件元数据和索引节点（inode）；文件的操作：建立，删除，打开，关闭，读，写文件的保护；文件的逻辑结构；文件的物理结构

（二）目录

目录的基本概念；树形目录；目录的操作；硬链接和软链接

（三）文件系统

文件系统的全局结构（layout）：文件系统在外存中的结构，文件系统在内存中的结构；外存空闲空间管理办法；虚拟文件系统；文件系统挂载（mounting）

【复习提示】本章内容较为具体，要注意对概念的理解。重点掌握**文件系统的结构及其实现、文件分配和空闲空间管理**等。要掌握文件系统的文件控制块、物理分配方法、索引结构、树形目录结构、文件共享原理、文件系统的布局、虚拟文件系统原理等。这些都是统考真题容易考查的内容。

一、基本概念

文件(file)



定义：是记录在**外存**上的，具有**符号名**的，在**逻辑上**具有**完整意义**的一组相关信息项的集合。

从用户的角度看，文件是**逻辑外存**的**最小分配单元**，即**信息(数据)**只能以文件的形式写入外存。

一、基本概念

文件的组成部分



sort.c

文件体：文件真实的内容。

文件说明(属性)：操作系统为了管理文件所用到的信息。

- 文件名称
- 文件类型
- 文件存储位置
- 文件大小
- 访问权限
- 时间、日期、用户标识
-

特性	文件内部标识符： inode	文件属性（如Windows 属性对话框）
物理存储位置	存储在文件系统 元数据区 （磁盘 固定位置）	部分属性存储在inode 中，部分属性（如注 释、标签）可能存储 在文件内容或扩展元 数据中
操作系 统依赖	由 文件系统 (ext4/NTFS) 直接管理	由 操作系统外壳 （如 Windows Explorer） 解 析显示
修改方 式	需调用 底层系统 调用（如chmod）	可通过 图形界面 直接 修改
不可变 部分	inode号、物理块 指针等永不改变	所有属性 对用户可见 且可修改

一、基本概念

文件的组成部分



文件体：文件真实的内容。

文件说明(属性)：操作系统为了管理文件所用到的信息。

- 文件名称
- 文件类型
- 文件存储位置
- 文件大小
- 访问权限
- 时间、日期、用户标识
-

文件目录



一、基本概念



文件的类型

❖ 分类的目的:

- 对不同文件进行区分和管理，提高系统效率；

文件的后缀

exe, com, bin:可执行文件
c, cc, cpp, java, pas:程序源代码
bat, sh: 命令文件
txt, doc: 文本数据， 文本文档
lib, dll: 库文件
zip, rar: 压缩文件
mpeg, mov, rm: 多媒体文件
.....

按照文件性质和用途

系统文件
库文件
用户文件

信息保存期限:

临时文件
档案文件
永久文件

文件的保护方式

只读文件
读写文件
可执行文件
不保护文件

一、基本概念



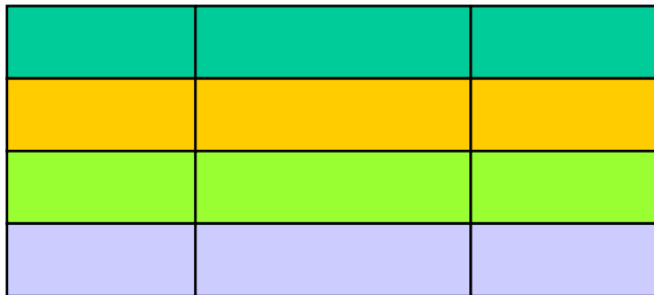
目前常用的文件系统类型

FAT16/	MS-DOS,win95 操作系统使用的文件系统。文件名采用 8.3 格式，支持分区最大 2G 。
FAT32	Windows 95 以上版本支持的文件系统，支持分区最大 32GB ，增加了对长文件名（最多到 255 个字符）支持。
NTFS	Windows NT 操作系统使用的文件系统。它具有较强的安全性，最大支持分区容量 2TB
Ext2/Ext3	Linux 系统中的文件系统，其特点为存取文件的性能极好，对于中小型的文件更显示出优势。单一文件大小上限为 2048GB ，而文件系统的容量上限为 16384GB 。
HPFS	OS/2 操作系统使用的文件系统。突破了 FAT 文件系统的一些限制，极大地改善了文件在大目录下的访问时间，支持 2TB 硬盘容量。
CDFS/UDF	大部分的光盘的文件系统，只能在 CD-R 或 CD-RW 上读取。
.....	

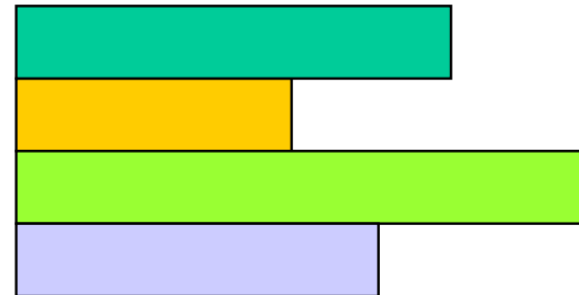
二、文件的结构

逻辑结构

- ❖ 定义：是用户所观察到的文件组织形式，它独立于物理特性，又称为文件组织 (File Organization)。
- ❖ 分类：
 - 有结构的记录式文件：由一个以上的记录构成。



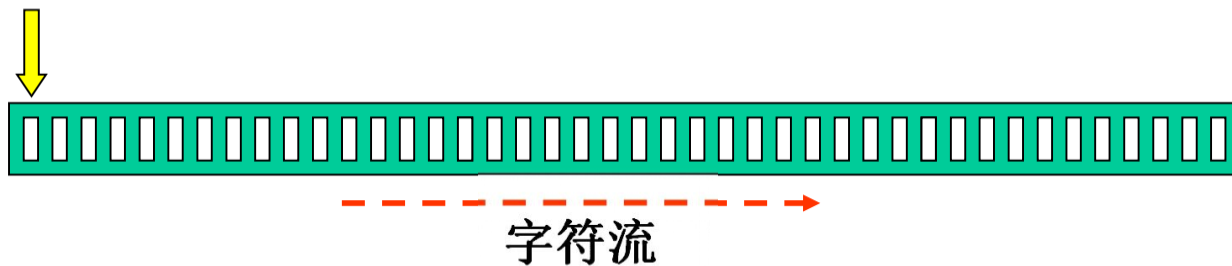
定长记录



变长记录

二、文件的结构

- **无结构的流式文件**：文件没有结构，由一串字符流构成。

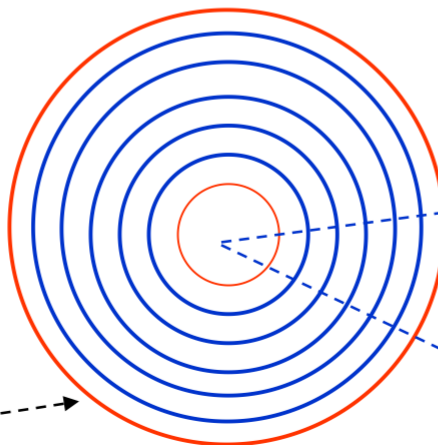
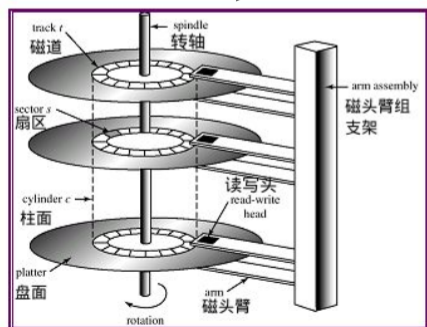


二、文件的结构



物理结构

- ❖ 定义：是指文件的内部组织形式，即文件在物理存储设备上的存放方法。



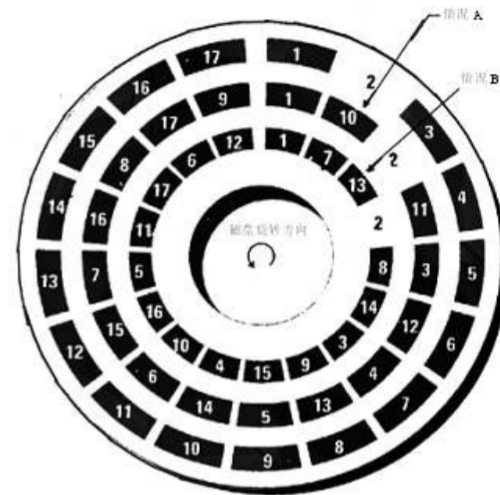
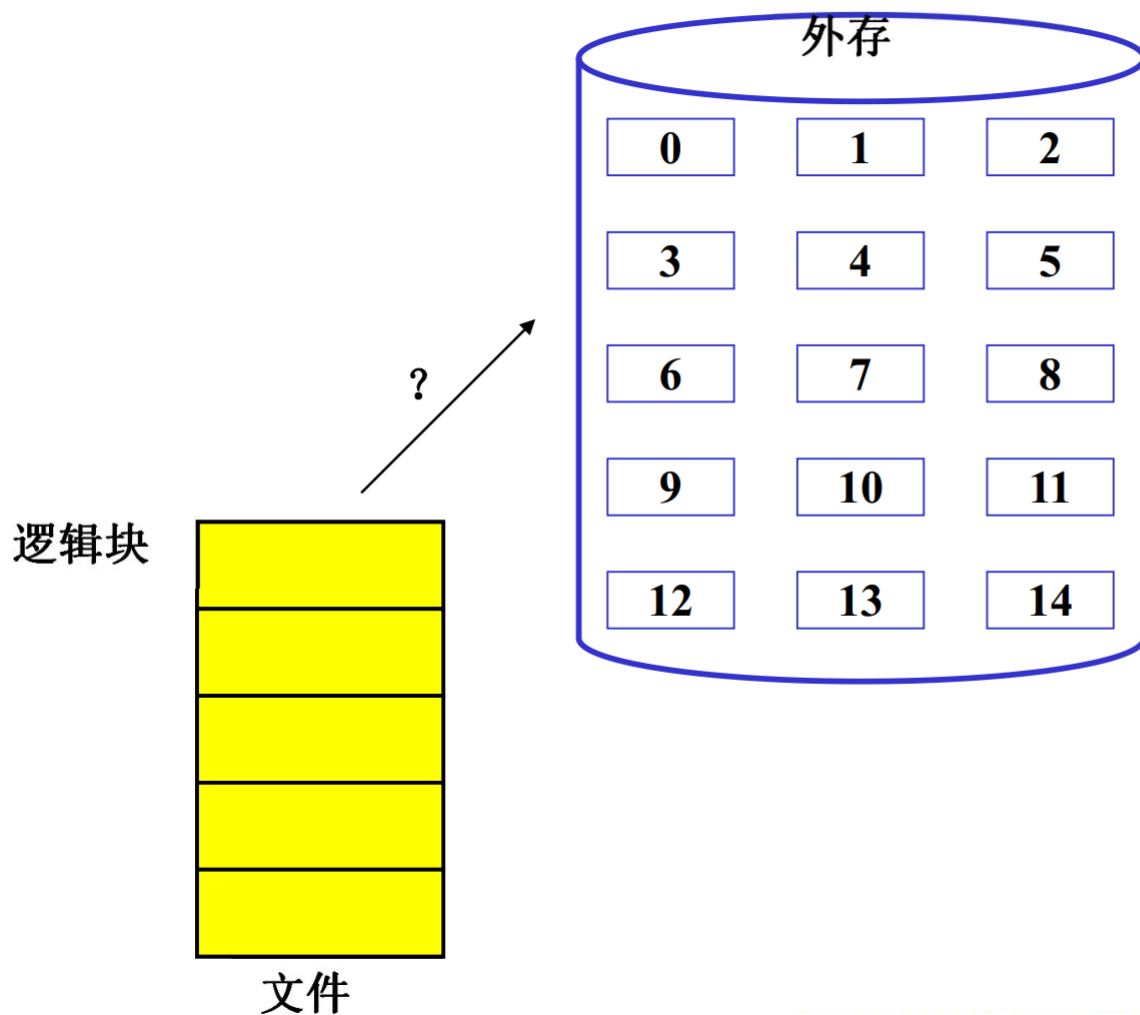
扇区：一般为600字节，其中512字节存放数据，其余存放控制信息。一个或多个扇区组成簇，称为物理块。

六、文件系统的性能问题

机械硬盘和固态硬盘的主要区别

特性	机械硬盘	固态硬盘
核心结构	机械部件（盘片、磁头）	闪存芯片 & 主控
工作原理	磁头在旋转盘片上移动读写	纯电子信号读写，无机械运动
速度	慢，受物理运动限制	极快，比机械硬盘快数倍至数十倍
抗震性	差，怕震动、摔落	强，无活动部件，更可靠
噪音	有运行噪音	完全静音
发热与功耗	较高	较低
重量	较重	较轻
价格	容量单价低，性价比高	容量单价高
寿命	理论寿命长，依赖机械磨损	有写入次数限制，但足够日常使用多年
数据恢复	损坏后可能部分恢复	损坏后极难恢复
操作系统影响	需要磁盘调度算法、易产生碎片	无需复杂调度、需TRIM指令、碎片影响小

二、文件的结构

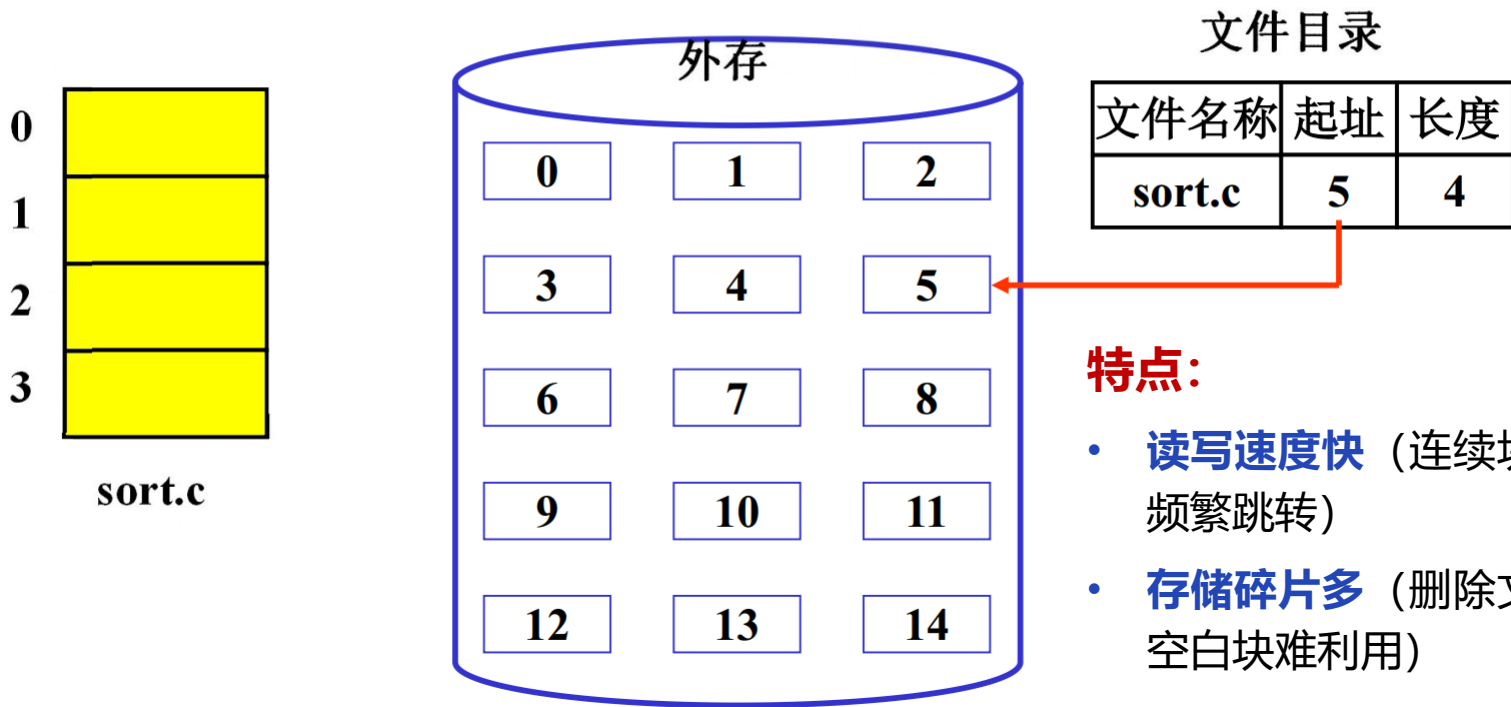


二、文件的结构

几种常见的物理存储方式

❖ 连续存储(顺序结构)

➤ 它将逻辑上连续的文件信息依次存放在编号连续的物理块上。



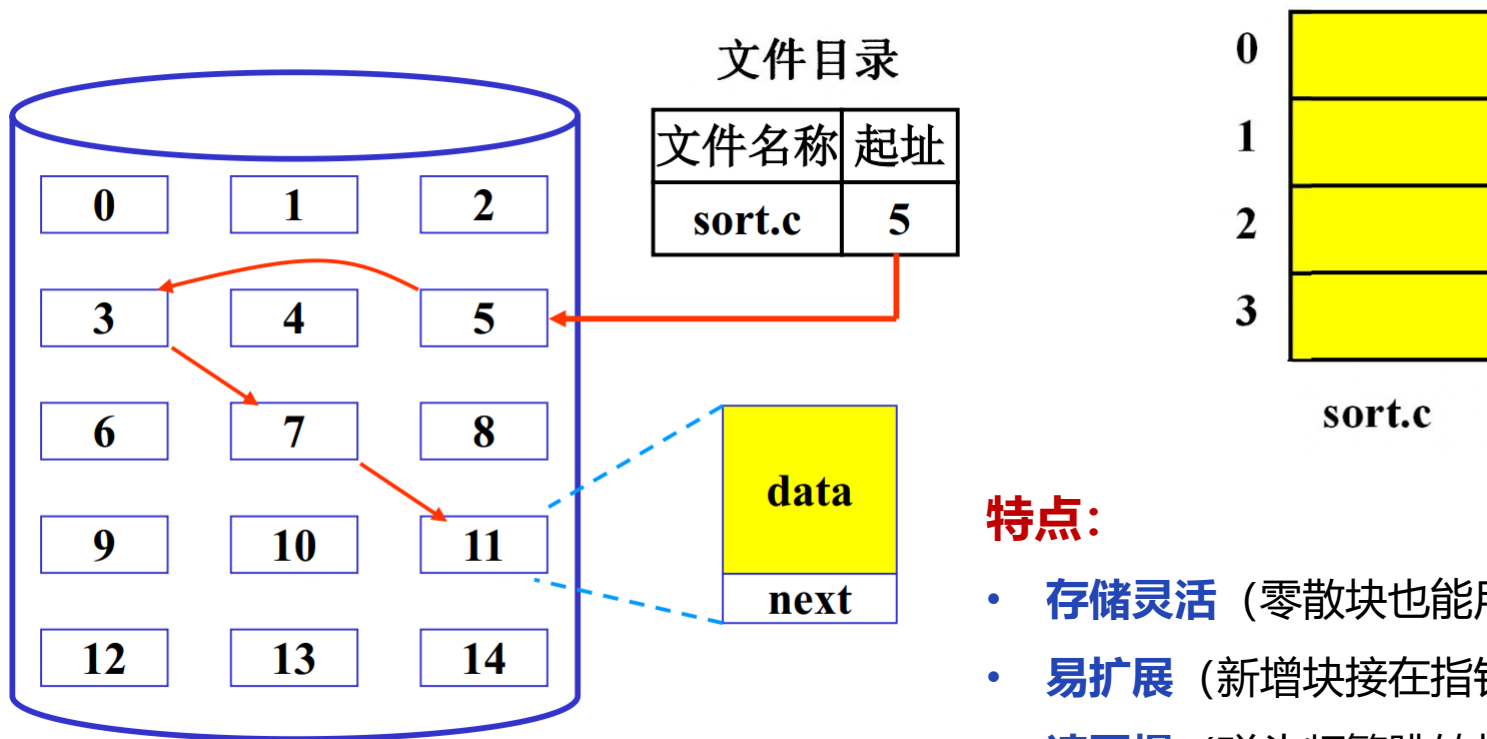
特点:

- **读写速度快** (连续块, 磁头不用频繁跳转)
- **存储碎片多** (删除文件后, 连续空白块难利用)
- **扩展难** (文件增大时, 后续连续块可能被占用)

二、文件的结构

❖ 链接结构

- 将逻辑上连续的文件信息存放在不连续的物理块上，每个物理块设有一个指针指向下一个物理块。



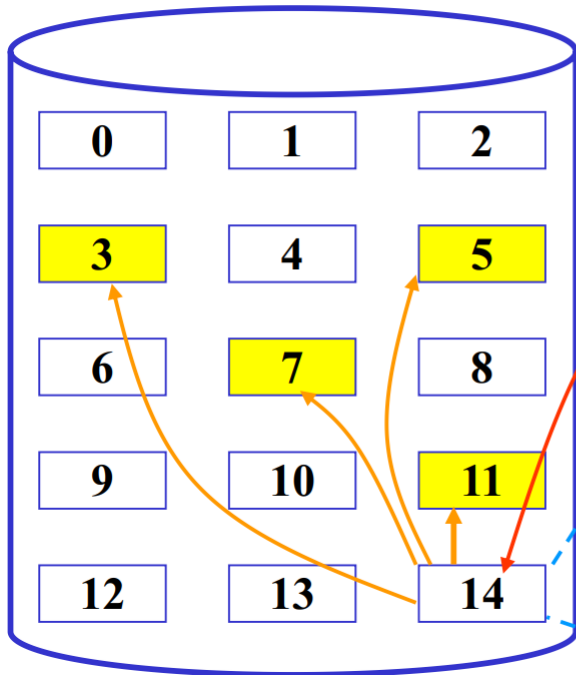
特点:

- **存储灵活** (零散块也能用)
- **易扩展** (新增块接在指针后)
- **读写慢** (磁头频繁跳转找指针)
- **可靠性差** (指针损坏, 文件易断裂)

二、文件的结构

❖ 索引结构

- 将逻辑上连续的文件信息(记录)存放在不连续的物理块中，系统为每个文件建立一个专用数据结构——**索引表**，索引表中存放文件的逻辑块号和物理块号的对应关系。

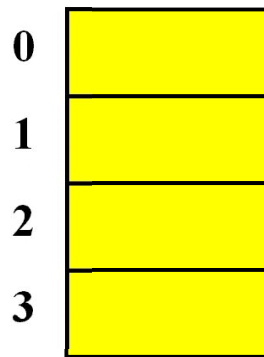


文件目录

文件名称	起址
sort.c	14

逻辑块号 物理块号

0	5
1	3
2	7
3	11



sort.c

特点:

- **存储灵活** (零散块高效利用)
- **易扩展** (改索引表就行)
- **读写快** (直接查索引定位块)
- **占额外空间** (存索引表)
- **索引表本身也需管理** (如太大分多级索引)

二、文件的结构

文件的存取方式

- ❖ 文件的存取方式是指读写文件存储器上的一个物理块的方法。
- ❖ 分类：
 - 顺序存取：指对文件中的信息按顺序依次读写的方式。
 - 随机读取
 - ① 直接存取法：允许用户随意存取文件中任意一个物理记录。
 - ② 按键存取法：根据文件中各记录的某个数据项内容来存取记录的，这种数据项称之为“键”。
- ❖ 文件结构、文件存取方式与文件存储介质的关系

存储设备	磁带	磁盘		
文件结构	连续	连续	串联	索引
存取方式	顺序	顺序、直接	顺序	顺序、直接

三、文件的目录



文件目录的内容



sort.c

文件体：文件真实的内容。

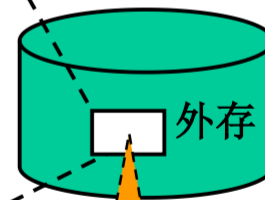
文件说明(属性)：操作系统为了管理文件所用到的信息。

- 文件名称
- 文件类型
- 文件存储位置
- 文件大小
- 访问权限
- 时间、日期、用户标识
-

FCB, 文件控制块



文件目录



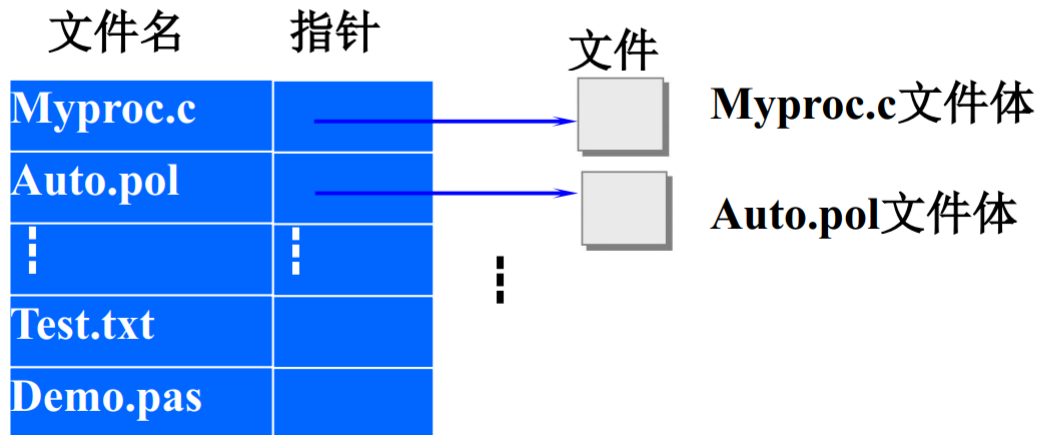
目录文件

三、文件的目录

✚ 文件目录结构的组织方式直接影响到文件的存取速度，关系到文件共享性和安全性，因此组织好文件的目录是设计文件系统的重要环节。

✚ 文件的目录结构分类

❖ 一级目录结构



优点：简单

缺点：

- 1、查找速度慢
- 2、不允许重名
- 3、不便于实现文件共享



单用户环境

三、文件的目录

❖ 二级目录结构

主目录MFD

(Master File Directory)

用户名	指针
A	26
B	32
⋮	⋮
D	103
E	58

文件名

指针

文件

Myproc.c

Auto.pol

⋮

Test.txt

Demo.pas

proc.mbd

Auto.pol

⋮

Setup.exe

Install.ini

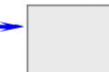
26

32

⋮

103

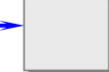
58



⋮



⋮



Myproc.c文件体

Auto.pol文件体

Demo.pas文件体

Proc.mbd文件体

Auto.pol文件体

Install.ini文件体

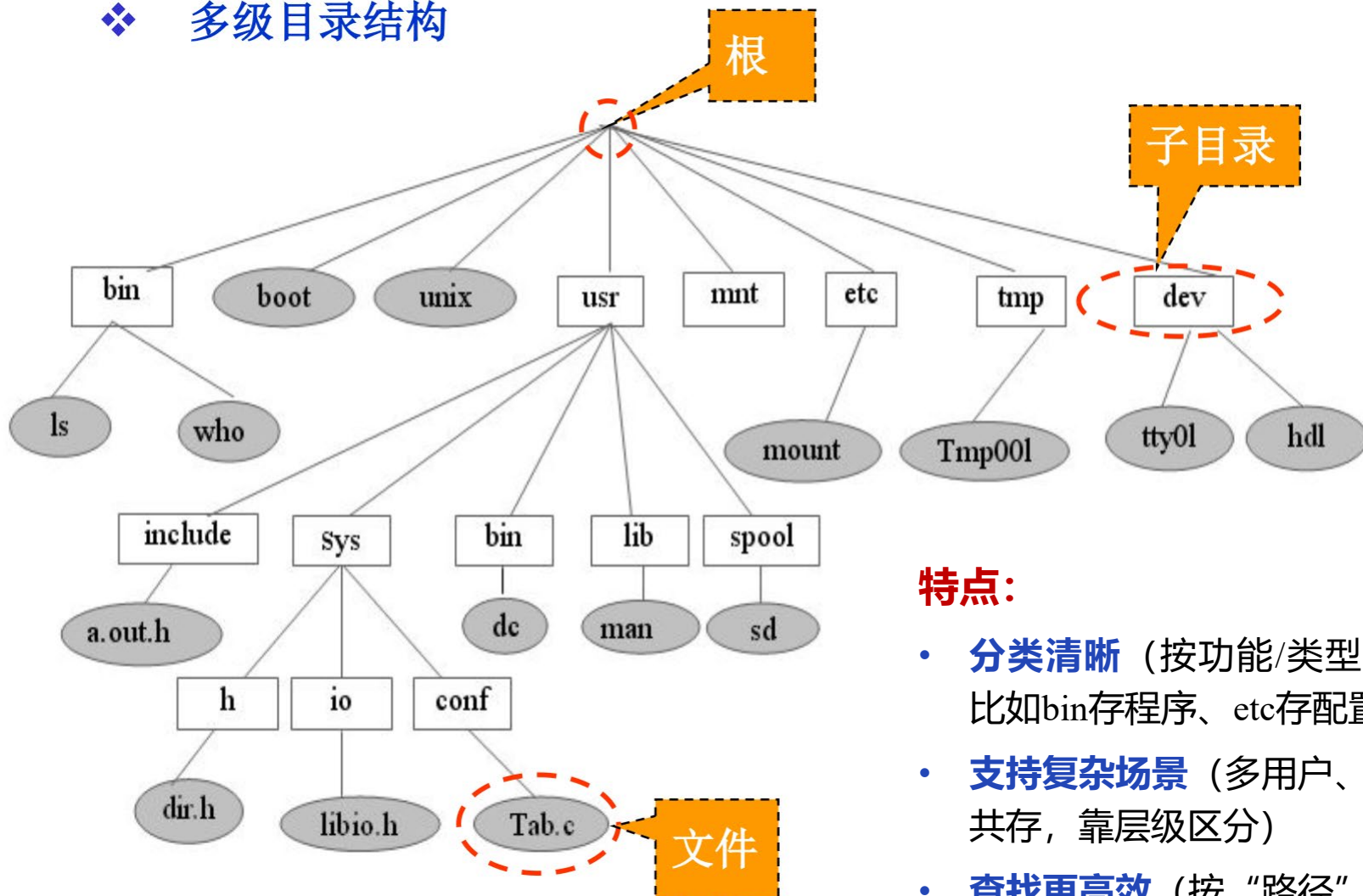
用户目录UFD

特点:

- **支持多用户** (不同用户可存同名文件, 因为分属不同用户目录)
- **查找变快** (先找主目录定位用户, 再在用户目录找文件, 范围缩小)

三、文件的目录

❖ 多级目录结构



特点:

- **分类清晰** (按功能/类型建子目录, 比如bin存程序、etc存配置)
- **支持复杂场景** (多用户、多类型文件共存, 靠层级区分)
- **查找更高效** (按“路径”定位, 比如usr/include/dir.h, 一步步缩小范围)

三、文件的目录

- **文件控制块 (File Control Block, FCB)**

- 用于**描述和控制文件的数据结构**，它至少应包括文件名和存放文件的盘物理地址
- 文件控制块的有序集合称为文件目录，即一个文件控制块FCB就是一个文件目录项。

- **FCB 包含的信息：**

- 文件基本信息：文件名，用户名，文件地址，文件长度，文件结构
- 存取控制信息：文件存取权限
- 管理信息：共享计数，文件的建立日期，保存期限，最后修改日期，最后访问日期

三、文件的目录

- **文件名：**标识一个文件的符号名，在每个系统中文件必须具有**唯一**的名字。
- **文件的物理地址：**因文件物理结构的不同而不同，因系统而异
 - 连续文件：文件的起始块号+文件总块数
 - MS-DOS：文件的起始簇号+文件总字节数
 - UNIX SV：文件所在设备的设备号、13个地址项、文件长度和文件块数等

三、文件的目录

- **文件目录**：文件控制块的有序集合
 - 文件目录是用于**检索文件**的，它是文件系统实现**按名存取**的重要手段，它的组织和管理应便于检索和防止冲突
 - **目录项**：构成文件目录的项目（**早期经典理论中：目录项就是FCB**）
- **目录文件**：为了实现对文件目录的管理，将文件目录以文件的形式保存在外存，这个文件就叫目录文件

三、文件的目录

- **路径名**：从**根目录出发**到某个文件的通路上**所有各级子目录名和该文件名的顺序组合**，中间可用 “/” 隔开
- 每个文件都有一个**唯一**的路径名，用户存取文件时必须给出文件的路径名。
- 文件系统根据用户指定的路径名**检索各级目录**，从而确定文件所在的位置。
 - Absolute path name (**绝对路径**)：从根目录开始到指定文件的路径；
 - Relative path name (**相对路径**)：从当前目录出发到指定文件的路径。
 - **如果文件就在当前目录中**，则存取文件时只要指出文件名就行，文件系统将在当前目录中寻找该文件。
 - **如果文件不在当前目录中**，但在当前目录的下级目录中，则可用相对路径名指定文件，文件系统就从当前目录开始沿着指定的路径查找该文件。
 - 使用**相对路径名**可以**减少查找文件所花费的时间**。

三、文件的目录

当前目录（工作目录、值班目录）

- 用户在一段时间内会经常访问一个子目录下的文件。
- 为了**提高效率**和**方便用户**，引进了“当前目录”的概念。
- 系统初始启动后，当前目录就是根目录。当前目录可根据需要任意改变。
- 当前目录一般存放在内存。

三、文件的目录

文件目录检索

- **目录检索：**用户给出文件名，按名寻找目录项
 - 根据路径名检索：
 - 全路径名：从根开始
 - 相对路径：从当前目录开始
- **文件寻址：**根据 FCB 中文件物理地址等信息，求出文件的任意记录或字符**在存取介质上的地址**

三、文件的目录

目录查询技术

- **哈希表算法**：目录项信息存在哈希表中
搜索时根据文件名**计算哈希值**
得到一个指向表中文件的指针
- **线性检索法**：即顺序检索法
利用给定文件名，在各级目录中顺序查找

三、文件的目录

目录查询技术

➤ 哈希表算法

假设哈希函数为： 文件名字符数 MOD 4

文件名	字符数	哈希值 (字符数 MOD 4)	文件地址
a.txt	5	1	0x1000
bc.jpg	6	2	0x1040
cat.png	7	3	0x1080
dog.jpg	7	3	0x10C0
readme	6	2	0x1100

哈希表结构

哈希槽位	存储的文件名 → 地址
0	(空)
1	a.txt → 0x1000
2	bc.jpg → 0x1040 readme → 0x1100
3	cat.png → 0x1080 dog.jpg → 0x10C0

三、文件的目录

目录查询技术

➤ 哈希表算法

哈希冲突：不同文件哈希值相等——系统会在哈希表的每个槽位中存储一个链表。当多个文件映射到同一个位置时，就把它们用链表连接起来。

哈希槽位	存储的文件名 → 地址
0	(空)
1	a.txt → 0x1000
2	链表头 → [bc.jpg → 0x1040] → [readme → 0x1100] → (NULL)
3	链表头 → [cat.png → 0x1080] → [dog.jpg → 0x10C0] → (NULL)

□ 查找 bc.jpg 的过程（有冲突时）：

1. 计算哈希值：字符数 6 MOD 4 = 2
2. 访问槽位 2：发现这里是一个链表
3. 遍历链表：第一个节点：bc.jpg → 找到，地址是 0x1040
4. 获取地址：通过地址 0x1040 访问文件数据

三、文件的目录

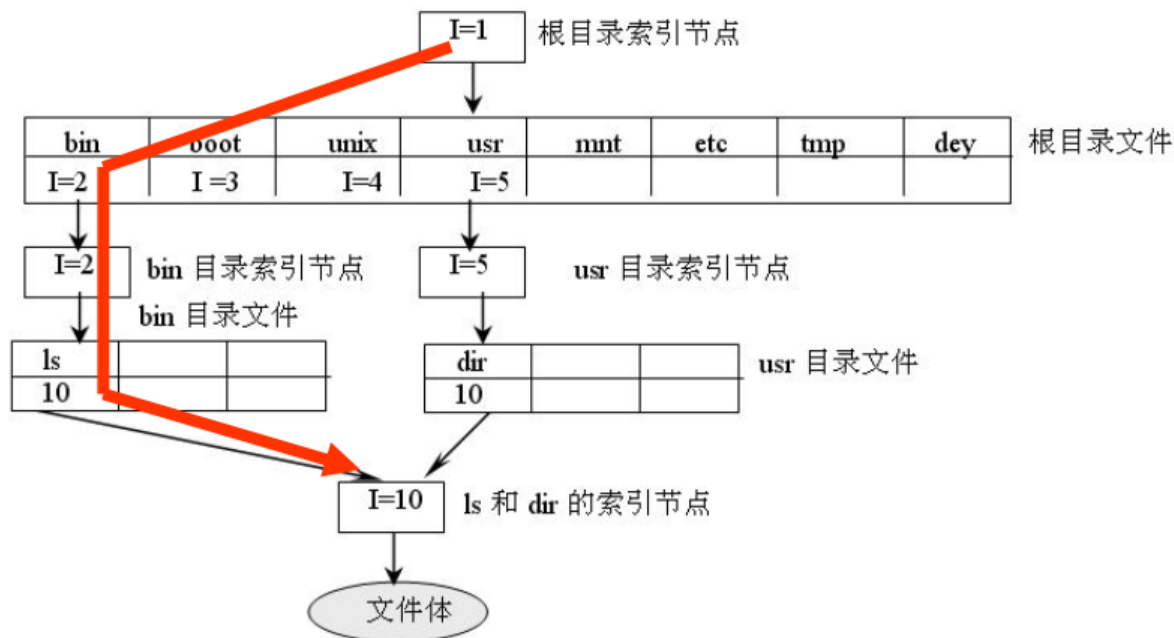
目录查询技术



线性检索法

- ❖ 为实现“按名存取”，UNIX系统按照如下步骤查找所需资源

例：查找/bin/ls



三、文件的目录

文件目录改进

- 为加快目录检索，可采用**目录项分解法**：把FCB分成两部分：
 - 符号目录项：文件名，文件号
 - 基本目录项：除文件名外的所有项目

	目录项	FCB
主要作用	定位文件 ：提供文件名到FCB或磁盘地址的映射，用于快速查找。	描述与管理文件 ：存储文件的所有元数据，是操作系统对文件进行任何操作（读、写、删）的依据。
包含的信息	通常很简单： 文件名 和 索引号/指针 。	非常详尽：文件大小、物理磁盘地址、创建/修改/访问时间、所有者、权限、文件类型等。
存储位置	存储在 目录文件 中。目录本身就是一个特殊的文件，里面存放着其包含文件的目录项。	现代系统（如Unix/Linux）中，FCB就是 inode ，存储在磁盘上单独的 inode区 。
数量关系	一对多 ：一个文件可以有多个目录项（即硬链接）。	一对一 ：一个文件有且仅有一个FCB（inode）。

三、文件的目录

文件目录改进

- **UNIX/Linux: inode (索引节点, 也称为磁盘索引节点)**

inode (index node, 索引节点) 是文件系统用于存储文件元数据的数据结构 (可以理解为一个文件的“身份证”或“档案卡”)。

inode 存储的是**除文件名以外的所有文件信息**, 例如: 文件的大小、文件的权限 (读、写、执行)、文件的所有者和所属组、文件的创建、修改、访问时间戳、**文件数据块在硬盘上的存储位置指针** (最重要的信息)、链接计数 (有多少个文件名指向这个inode) 等。

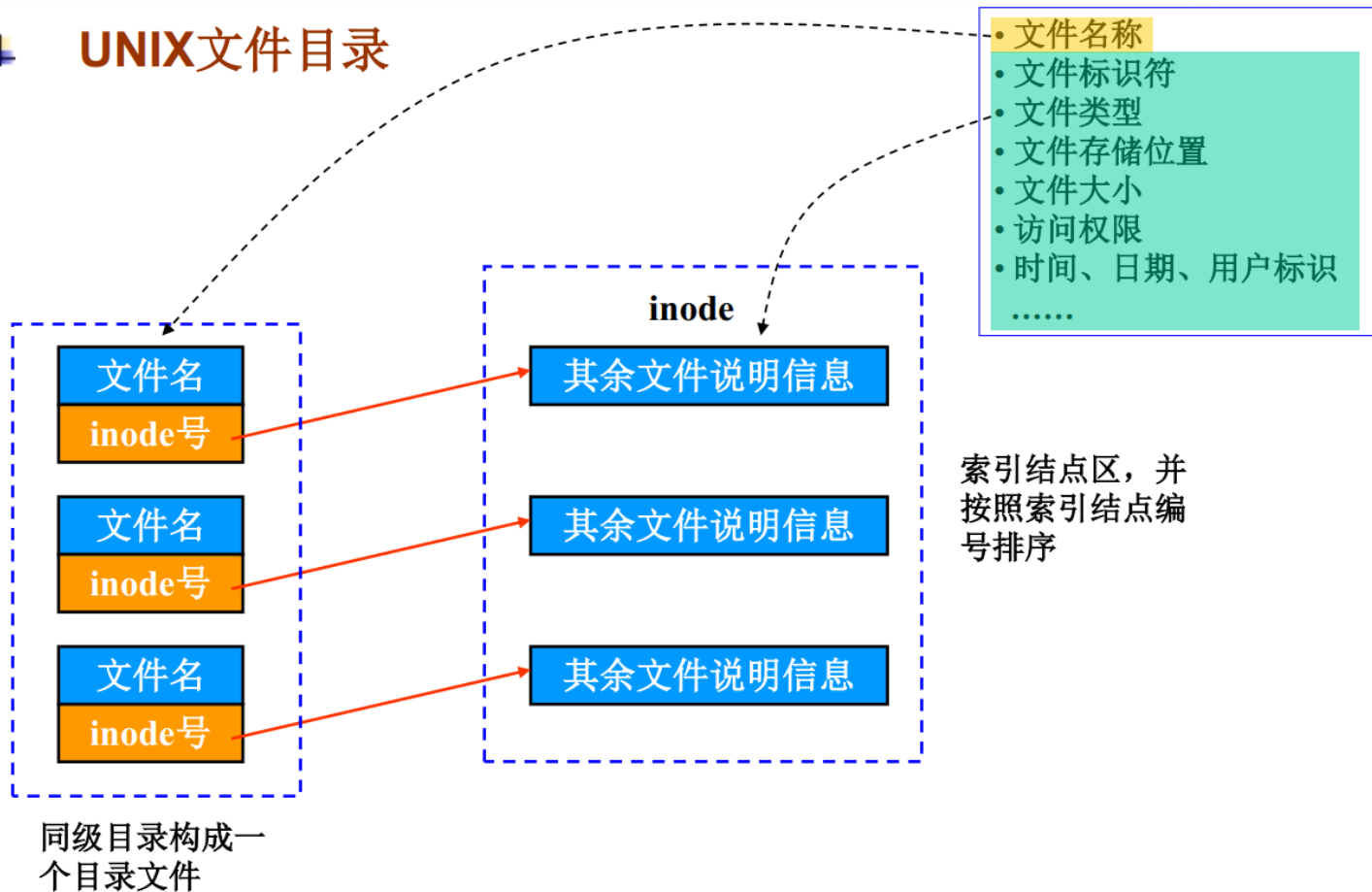
例如bin目录的inode: 在根目录的文件体里找到 bin这一项时, 会关联一个inode编号。这个inode里的指针, 则指向了存储/bin目录文件体的数据块。

三、文件的目录

文件目录改进



UNIX文件目录



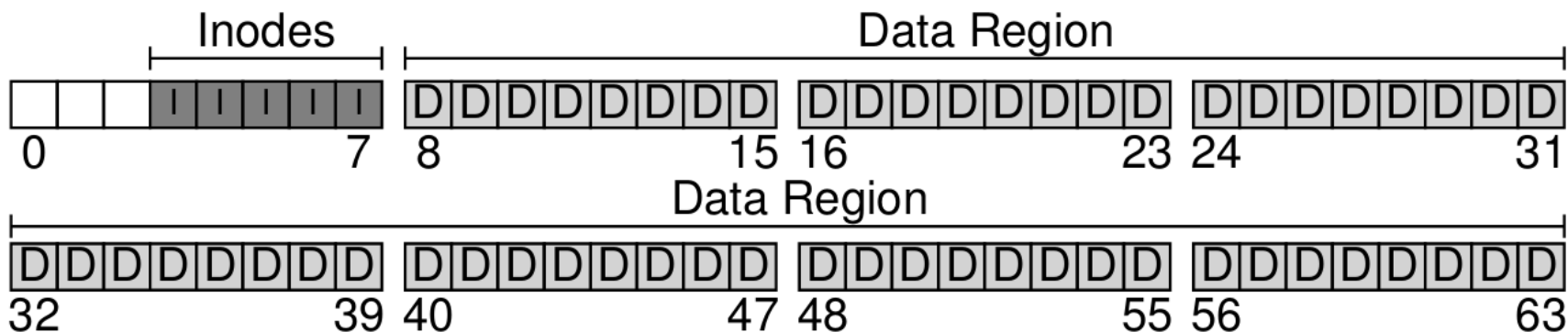
三、文件的目录

UNIX的目录结构

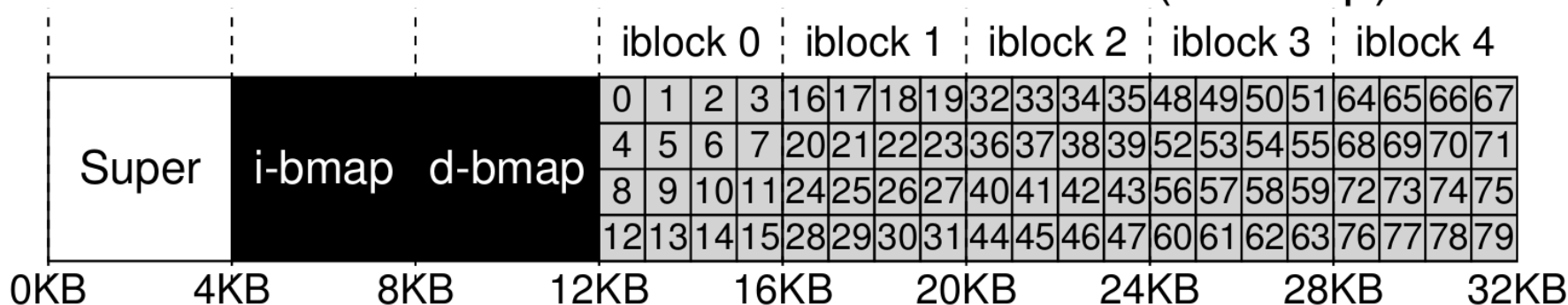
- UNIX 采用**文件名**和**文件说明分离**的目录结构
- 每个文件有一个存放在**磁盘索引节点区**的**索引节点**，称为**磁盘索引节点**，它包括以下内容：
 - 文件**主标识符**和**同组用户标识符**；
 - **文件类型**：是普通文件、目录文件、符号连接文件或特别文件（又分块设备文件或字符设备文件）
 - 文件主，同组用户和其它人对文件**存取权限**（读R、写W、执行X）
 - 文件的**物理地址**，用于UNIX直接、间接混合寻址的13个地址项di_addr[13]
 - 文件**长度**（字节数） di_size
 - 文件**链接数**di_nlink
 - 文件**最近存取**和**修改时间**等

三、文件的目录

- 给定文件路径名为 /usr/ast/mbox, 检索

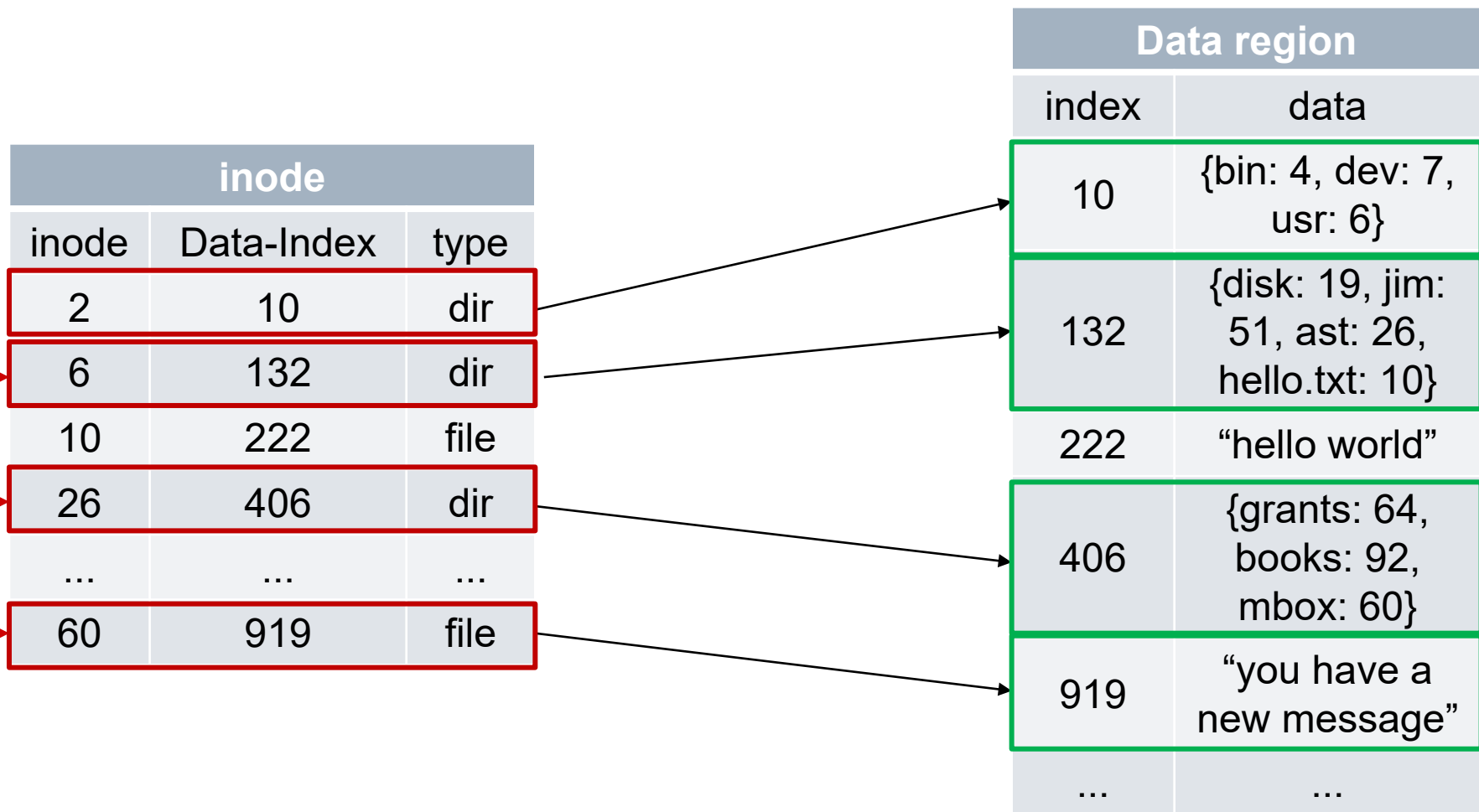


The Inode Table (Closeup)



三、文件的目录

- 给定文件路径名为 `/usr/ast/mbox`，检索过程如下



三、文件的目录

查找 /usr/ast/mbox 的步骤

- 首先，系统应先**读入第一个文件分量名usr**，用它与根目录文件(或当前目录文件)中各个目录项的文件名顺序地进行比较，从中找到匹配者，并**得到匹配项的索引结点号6**，再从6号索引结点中**得知usr目录文件放在第132号盘块中**，将它读入内存；
- 然后，系统再将路径名中的**第二个分量名ast读入**，用它与放在第132号盘块中的第二级目录文件中的各目录项内的文件名顺序进行比较，又找到匹配项，从中得到ast的目录文件放在**第26号索引结点**中，再从26号索引结点中**得知 /usr/ast 是存放在406号盘块中**，再读入第406号盘块；
- 尔后，系统又将该文件的**第三分量名mbox读入**，用它与第三级目录文件 /usr/ast中各目录项中的文件名进行比较，最后得到 /usr/ast/mbox的**索引结点号为60**，即在60号索引结点中存放了指定文件的**物理地址**。目录查询操作至此结束；
- 如果在顺序查找过程中，发现一个文件分量名未能找到，则应停止查找并返回“文件未找到”信息。

四、文件系统的实现 - 文件空间的分配和管理

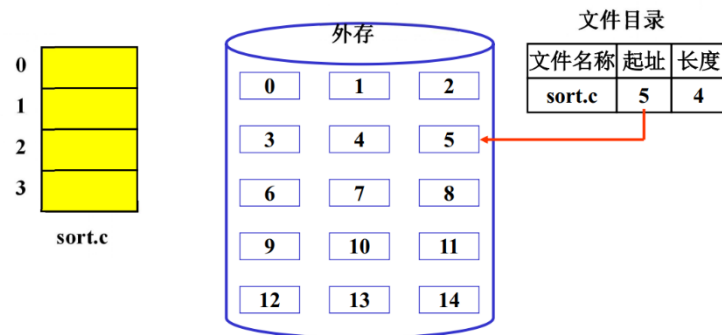
✓ 连续分配

✓ 链接分配 (显式、隐式)

✓ 索引分配

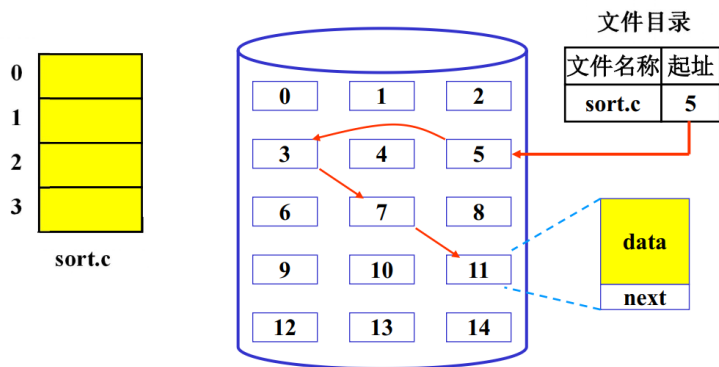
❖ 连续存储(顺序结构)

- 它将逻辑上连续的文件信息依次存放在编号连续的物理块上。



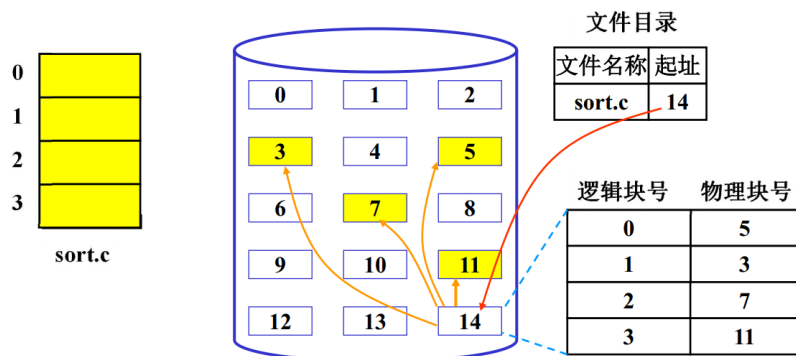
❖ 链接结构

- 将逻辑上连续的文件信息存放在不连续的物理块上，每个物理块设有一个指针指向下一个物理块。



❖ 索引结构

- 将逻辑上连续的文件信息(记录)存放在不连续的物理块中，系统为每个文件建立一个专用数据结构——索引表，索引表中存放文件的逻辑块号和物理块号的对应关系。



四、文件系统的实现 - 内存中所需表目

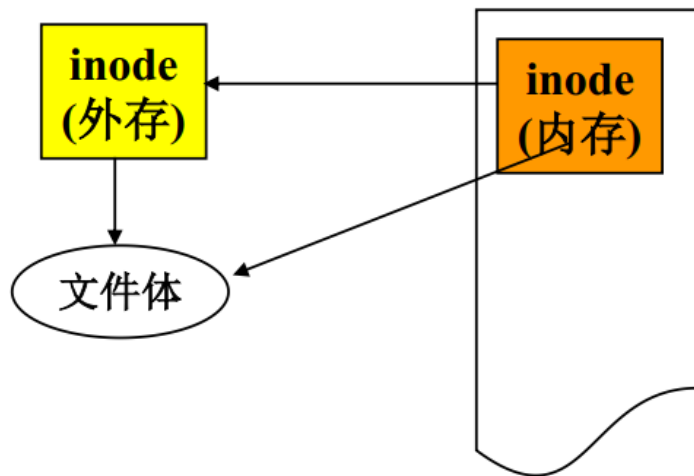


打开文件机构

- ❖ 操作系统在内存设置了一个非常精炼的文件机构，它不是外存文件管理机构的全部映象，而是存储最近正在使用文件的相关信息。
- ❖ 文件打开后由内存的一套管理机构管理，关闭时退出管理机构，所以将这种文件管理机构称为**打开文件机构**。
- ❖ 打开文件机构的结构：
 - 内存文件控制块
 - 系统打开文件表
 - 用户打开文件表

四、文件系统的实现 - 内存中所需表目

❖ 内存文件控制块



内存I节点结构与外存I节点结构基本相同，只是略微调整了一些内容，这里我们重点关注：

i_dev //文件所在设备号

i_number //对应外存的I节点号

• 内存inode的生命周期（打开→操作→关闭）

打开文件时：系统先查“内存inode表”，看有没有当前文件的内存inode；如果没有，分配一个空闲内存inode，把外存inode的关键信息复制进来（比如设备号、权限），并记录i_number（关联外存inode）；

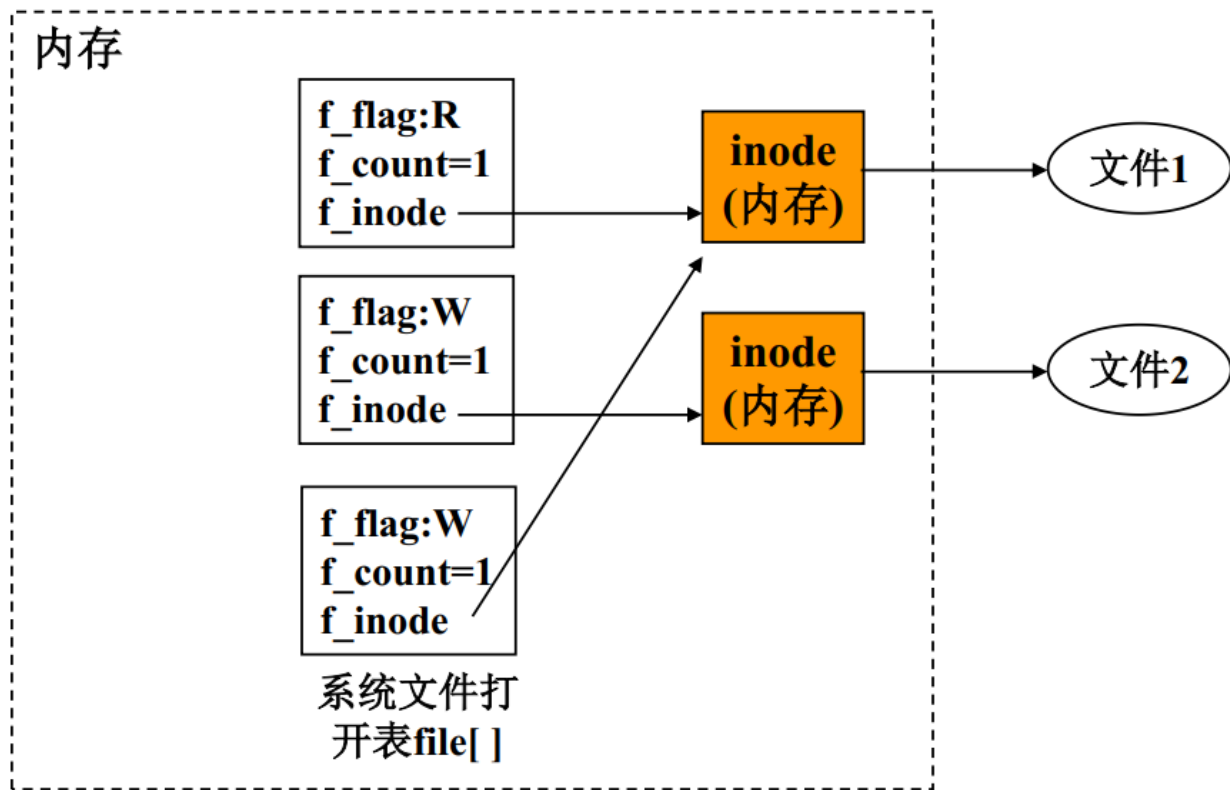
操作文件时：读写文件的“控制信息”（比如修改权限、记录访问时间），直接操作内存inode（内存读写快，不用每次读磁盘）；

关闭文件时：如果内存inode被修改过（比如权限变了、内容更新了），同步更新外存inode（把内存里的新信息写回磁盘），保证外存inode的数据最新。

四、文件系统的实现 - 内存中所需表目

❖ 系统打开文件表

- 系统打开文件表用于记录所有打开文件的控制信息。



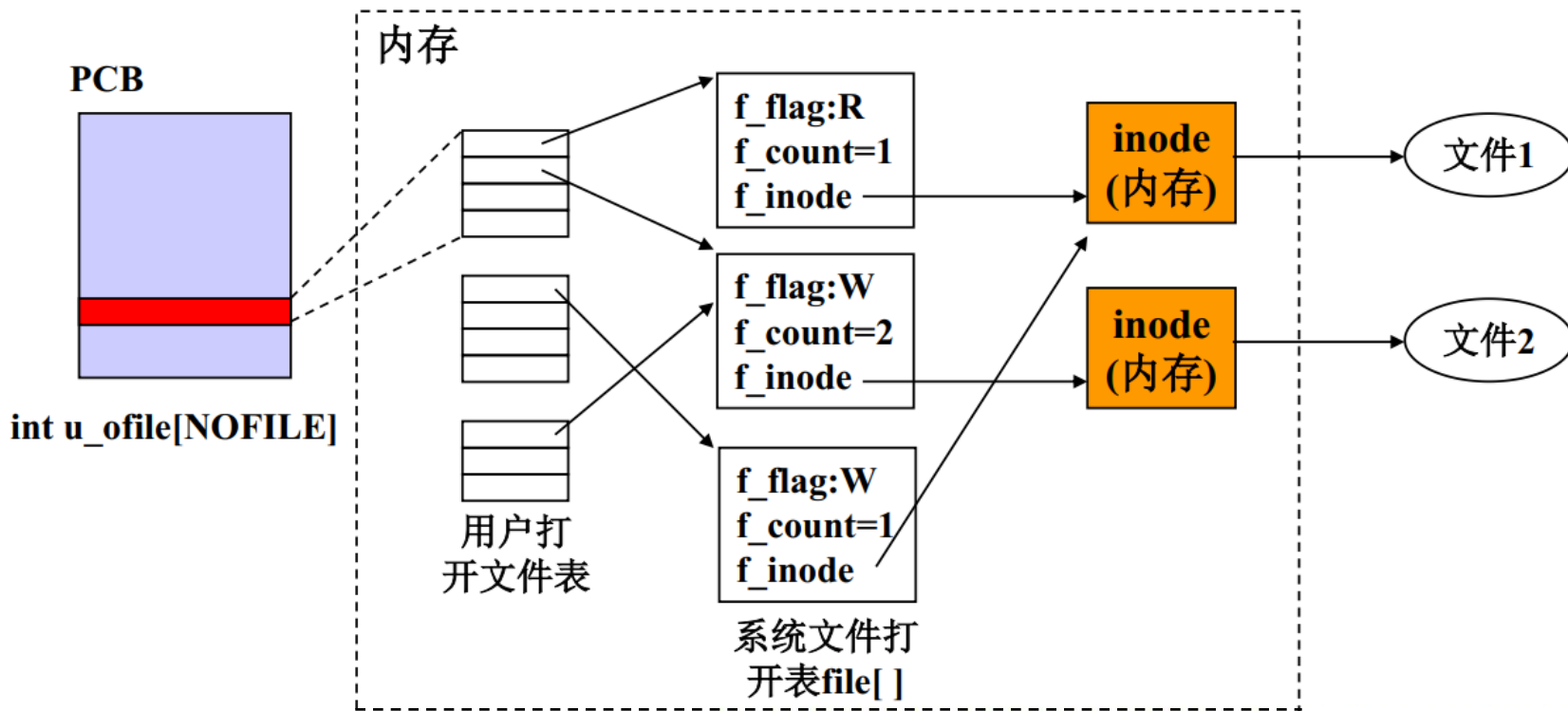
作用:

- **共享复用:** 多个进程打开同一文件时，共享“系统打开文件表”的记录，不用重复加载内存inode，节省资源
- **冲突控制:** 通过f_flag限制文件操作（比如一个进程写文件时，标记f_flag=W，其他进程只能读），避免“同时写文件导致数据混乱”
- **资源回收:** 靠f_count精准控制“文件何时释放资源”（引用计数为0时，才真正关闭文件，回收内存inode）

四、文件系统的实现 - 内存中所需表目

❖ 用户打开文件表

- 文件通常是通过进程来操作的，因此每一个文件都有一个用户(进程)。同样每一个进程可打开多个文件。
- 在操作系统中，每一个进程都有一张打开文件表——用户打开文件表。



四、文件系统的实现 - 内存中所需表目

❖ 用户打开文件表

➤ 流程示例（以“进程打开/操作文件”为例）

- **进程打开文件：**进程调用open()→系统分配/复用“系统打开文件表项”→在用户打开文件表新增记录，关联“系统表项”和“内存inode”，标记读写模式（如f_flag:R读、f_flag:W写）
- **进程操作文件：**进程调用read()/write()→通过用户打开文件表找到关联的“系统表项”和“内存inode”→直接操作“内存inode”（内存读写快），或通过“系统表项”的引用计数、权限判断，保证操作安全（如写操作需f_flag:W）
- **进程关闭文件：**进程调用close()→在用户打开文件表删除记录→系统表项的“引用计数（f_count）减1”→若计数为0，释放“内存inode”并同步外存

➤ 作用（解决多进程文件管理问题）

- **进程独立性：**每个进程的“用户打开文件表”独立，多进程操作同一文件时，可各自记录权限（如进程A读、进程B写）
- **资源共享：**通过“系统打开文件表”的引用计数，实现“多进程共享文件”（如3进程读文件1，系统表项f_count=3，避免重复加载内存inode）
- **高效读写：**内存inode作为“缓存”，减少磁盘IO，提升文件操作效率（进程操作时直接读内存元数据）

四、文件系统的实现 - 内存中所需表目

✚ 打开文件机构

- ❖ 操作系统在内存设置了一个非常精炼的文件机构，它不是外存文件管理机构的全部映象，而是存储最近正在使用文件的相关信息。
- ❖ 文件打开后由内存的一套管理机构管理，关闭时退出管理机构，所以将这种文件管理机构称为**打开文件机构**。
- ❖ 打开文件机构的结构：
 - 内存文件控制块
 - 系统打开文件表
 - 用户打开文件表

□ **内存文件控制块**存文件元数据，是**基础**；**系统打开文件表****全局记录**所有打开文件的控制信息，关联内存文件控制块；**用户打开文件表**为**进程专属**，关联系统打开文件表，三者从**文件元数据**、**系统全局**到**进程专属**，**分层协作**管理打开文件

四、文件系统的实现

对文件的操作

- **建立文件:**

- **建立**文件的FCB
- **分配**必要的外存空间
- 在文件目录中为之建立一个**目录项**，目录项中**记录**新文件的文件名及其在外存的地址等属性，返回一个文件描述符

- **删除文件:**

- 从目录中**找到**要删除文件的目录项
- 使之成为**空闲**目录项
- **回收**该文件所占用的存储空间

四、文件系统的实现

对文件的操作

- **读文件**：把文件中的数据**从外存读入内存**的用户区
 - 查找目录，**找到**指定文件的**目录项**，从中得到被读文件在外存的地址
 - 从外存将数据**读入**内存
- **写文件**：
 - **查找**目录，找到指定文件的目录项
 - 利用目录中的文件**指针**将信息**写入**文件

四、文件系统的实现 – 外存空间管理



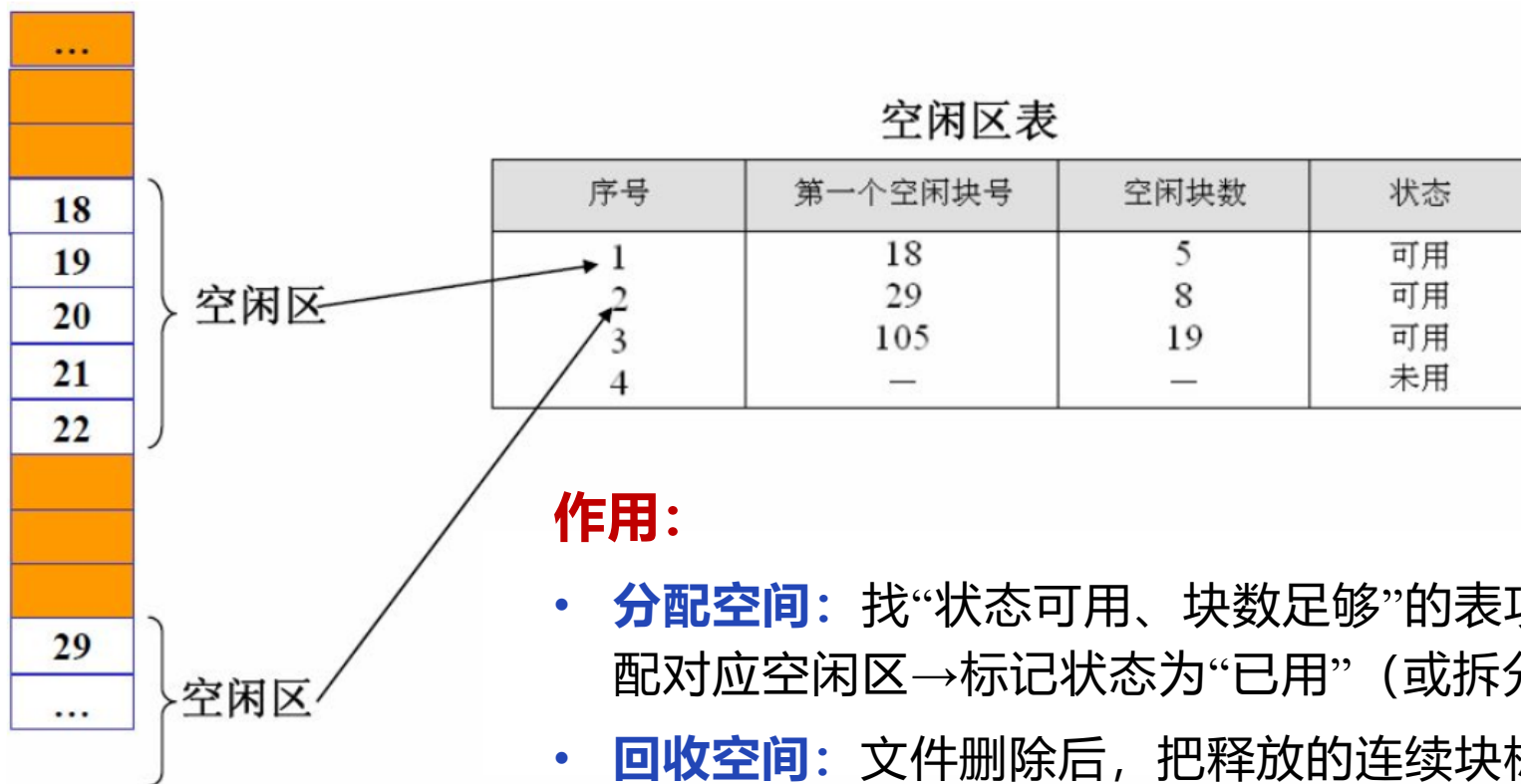
概述

- ❖ 外存具有较大的存储空间，且被多用户共享，用户执行程序经常要在磁盘上存储、删除文件，因此，文件系统必须对磁盘空间进行有效管理。
- ❖ 外存空闲空间管理的数据结构通常称为磁盘分配表(**disk allocation table**)。常用的空闲空间的管理方法有：
 - 空闲区表
 - 位示图
 - 空闲块链

四、文件系统的实现 – 外存空间管理

空闲区表

- ❖ 将外存空间上一个连续未分配区域称为“空闲区”。操作系统为磁盘外存上所有空闲区建立一张空闲表，每个表项对应一个空闲区，空闲表中包含序号、空闲区的第一个块号、空闲块的块数等信息。



四、文件系统的实现 – 外存空间管理

✚ 位示图

- ❖ 这种方法是在外存上建立一张位示图(bitmap)，记录文件存储器的使用情况。每一位对应文件存储器上的一个物理块，取值0和1分别表示空闲和占用。

第 1 字	1	0	1	0	0	...	1	1
第 2 字	0	1	1	1	0	...	0	1
第 3 字	1	1	1	1	0	...	1	0
⋮								
第 n 字	0	0	0	1	1	...	0	0

作用：

- **分配空间：** 遍历位示图，找值为0的位→分配对应物理块→标记该位为1
- **回收空间：** 文件删除后，找到对应物理块的位→标记为0，释放空间

特点：

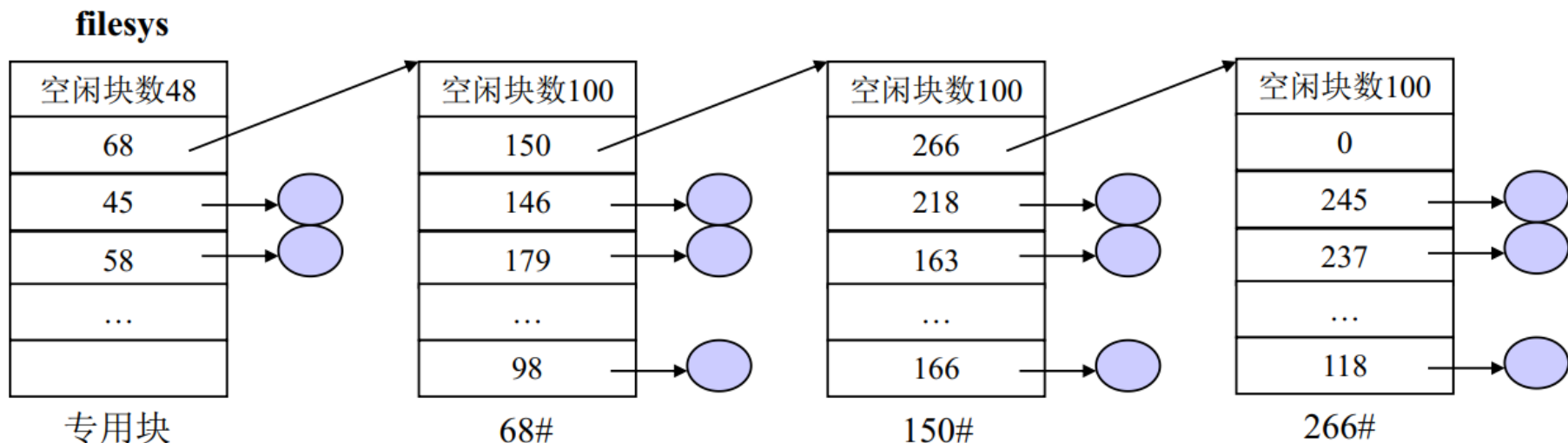
- **紧凑高效：** 用二进制位存储状态，极大节省空间（比如1KB位示图可管理8192个物理块）
- **查找方便：** 快速定位空闲块，适合大规模外存管理（如硬盘、SSD）

四、文件系统的实现 – 外存空间管理

成组链接法 —— 空闲块链法的变体

- ❖ 将空闲块分成若干组，每100个空闲块为一组。每组的第一个空闲块登记了下一组空闲块的物理盘块号和本组空闲块总数。

空闲块索引表



可以将整个空闲块管理系统想象成一个只有一个入口的仓库。这个入口就是内存中的“**专用块**”。

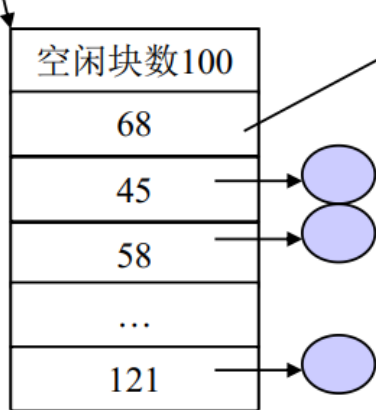
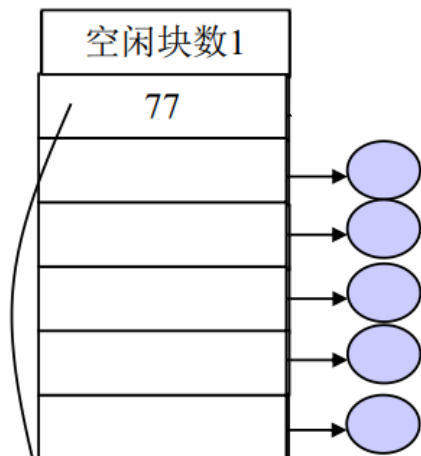
- **唯一操作入口**：所有分配与回收仅与内存中的“专用块”交互
- **全局索引根源**：通过它可索引并管理磁盘上的全部空闲块组

所以，从功能和操作的角度看，有且只有一个活跃的、起作用的“空闲块索引表”。

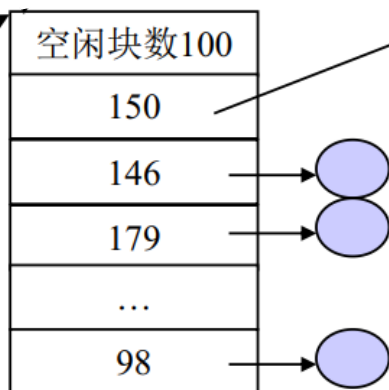
四、文件系统的实现 – 外存空间管理

❖ 释放算法

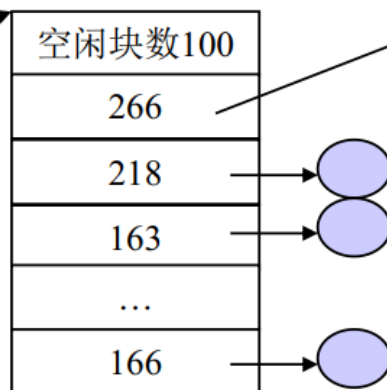
空闲块索引表
filesys



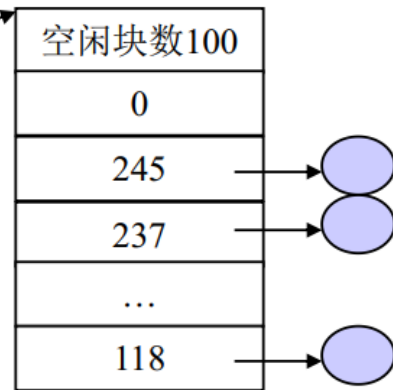
77#



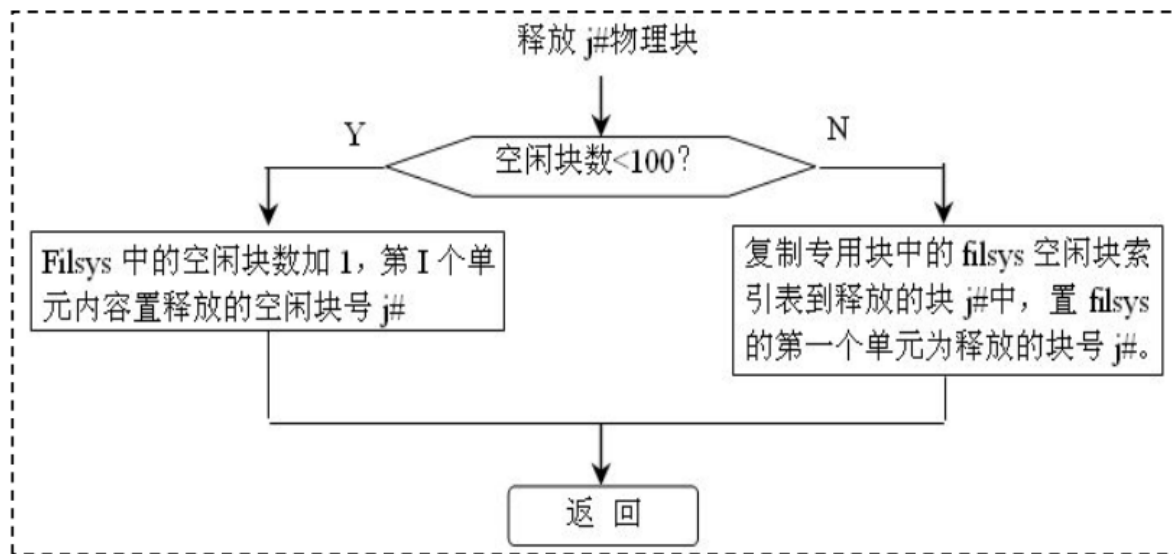
68#



150#

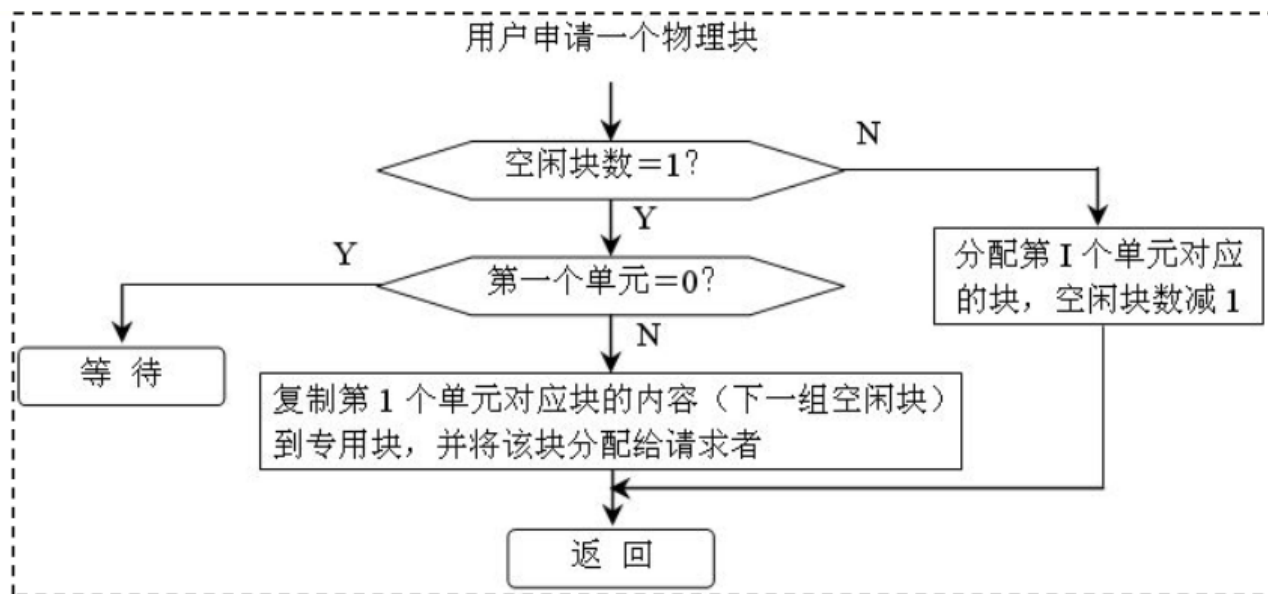


266#

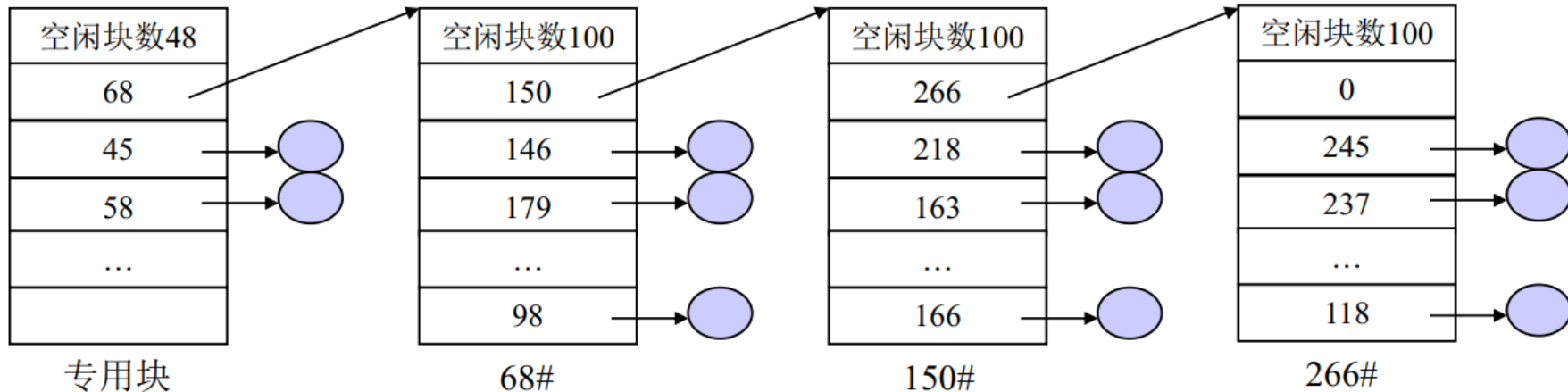


四、文件系统的实现 – 外存空间管理

❖ 分配算法



空闲块索引表
filesys



四、文件系统的实现 – 外存空间管理

三种外存空间管理方式的主要应用场景及在现代操作系统中的地位

数据结构	实际主要应用场景	现代操作系统中的地位与演进
空闲区表	传统可变分区内存管理	基本淘汰。 因其无法有效处理外部碎片，已被基于分页的现代内存管理机制所取代。
位示图	物理内存页框的管理； 部分文件系统的磁盘空间管理	主流。 作为分页内存管理的基石，因其空间开销极小、查找效率高，是管理物理内存的首选方案。
成组链接法	文件系统磁盘空间管理	主流方案之一。 通过将空闲块分组管理，将大量I/O操作转化为对少量“索引块”的操作，在效率与实现复杂度之间取得了绝佳平衡。

五、文件共享

✚ 共享的含义

- ❖ 文件共享是指不同用户或进程使用同一文件，实际上文件的实体只有一个。

✚ UNIX文件系统的特点

- ❖ 文件名(目录项)和文件说明(inode)分离——有利于实现共享。

✚ 静态共享

❖ 静态的含义

- 一个文件同时属于多个文件目录项，但实际上文件仅仅只有一处物理存储，这种多个文件目录项对应一个文件实体的多对一的关系叫做文件链接。

❖ 特点：

- 静态共享中这种关系不管文件此时是否在被使用，都存在。

五、文件共享



静态共享的实现

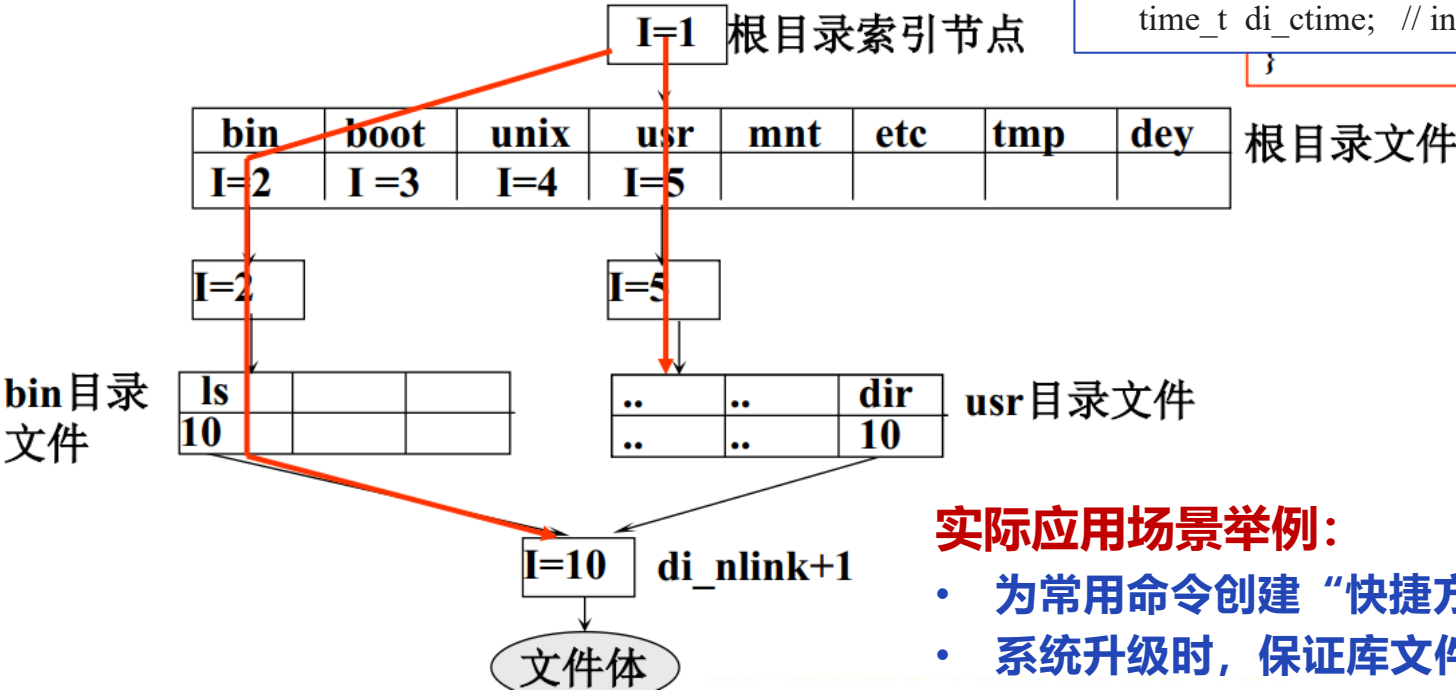
❖ 硬链接

➤ UNIX中文件链接的系统调用

`link(oldname, newname);`

例如: `link("/bin/ls", "/usr/dir");`

```
struct inode{
    ushort di_mode; // 文件权限、类型
    short di_nlink; // 链接数 (多少个文件名指向该 inode)
    ushort di_uid; // 用户 ID
    ushort di_gid; // 组 ID
    off_t di_size; // 文件大小
    char di_addr[40]; // 存储文件数据磁盘块地址
    time_t di_atime; // 最后访问时间
    time_t di_mtime; // 最后修改时间
    time_t di_ctime; // inode 变动时间 };
```



实际应用场景举例:

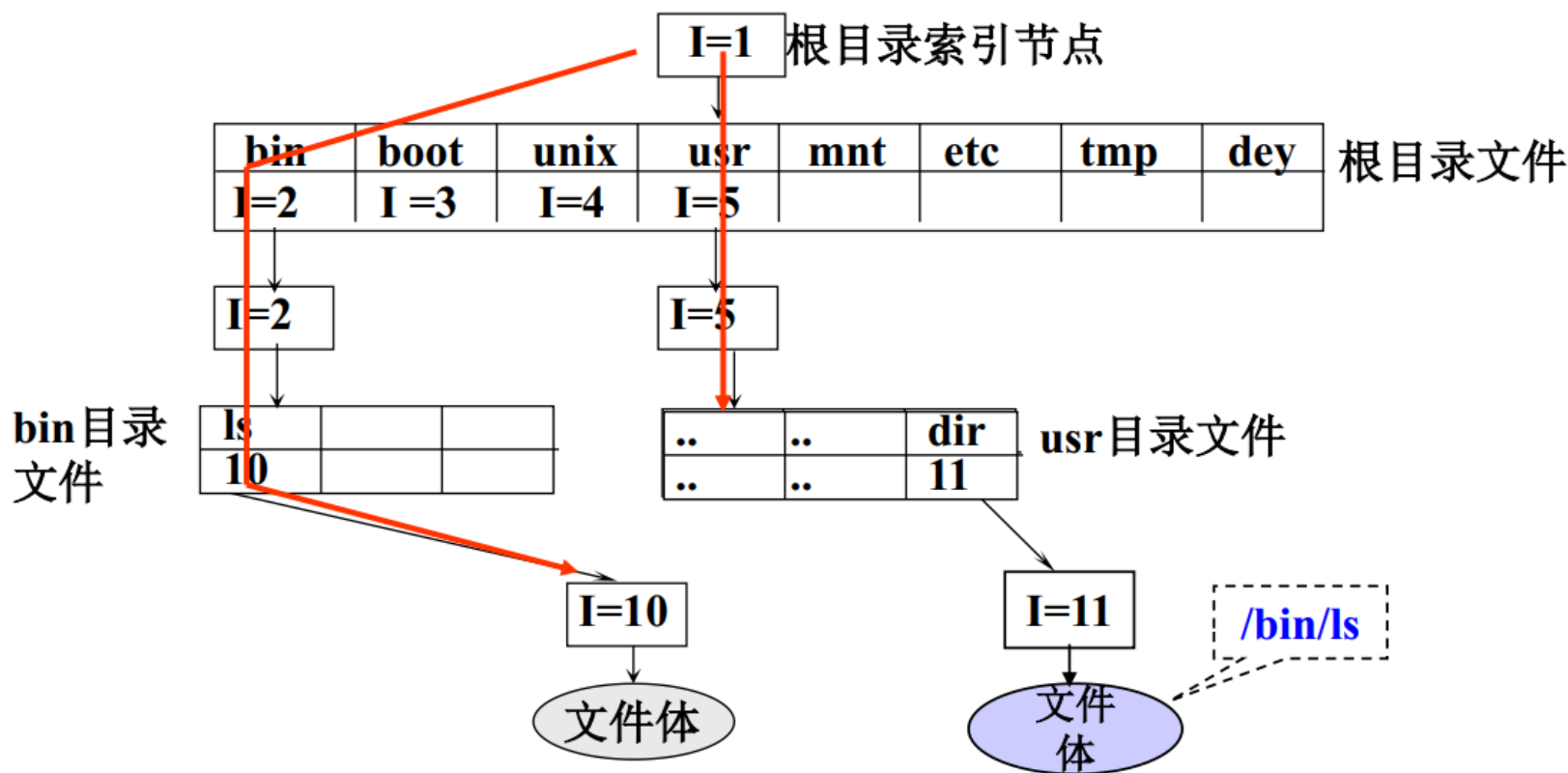
- 为常用命令创建“快捷方式” (无需复制文件)
- 系统升级时, 保证库文件和命令的向后兼容性 (旧的应用程序仍然会在代码里寻找并链接原来的库路径)

五、文件共享

❖ 软连接

➤ 也称**符号链接**，符号链接文件的内容为被链接文件的路径名。

举例: `ln -s /bin/ls /usr/dir`



五、文件共享

✚ 动态共享

❖ 动态的含义

- 出现在进程共享文件时，伴随着进程的生成而存在，进程的终止而消失。

❖ 举例

P1进程代码：

```
fd1=open("/etc/test", o_RDONLY);  
fd2=open("pocal", o_WRONLY);
```

P2为P1的子进程，代码如下：

```
fd3=open("/etc/testexa", o_RDONLY);
```

P3执行如下代码：

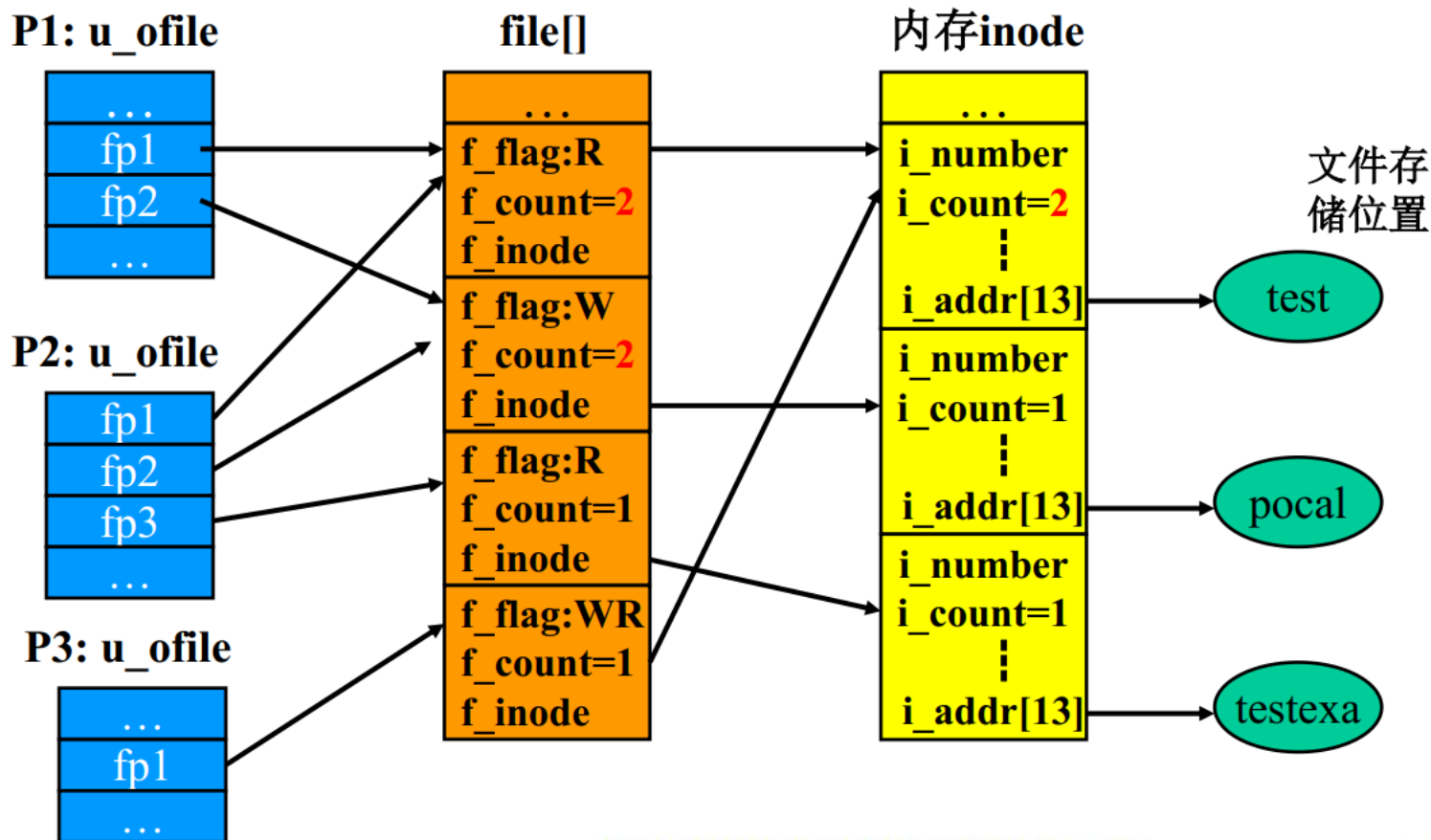
```
fd1=open("/etc/test", o_RDWR);
```

五、文件共享

```
fd1=open("/etc/test", o_RDONLY);  
fd2=open("pocal", o_WRONLY);
```

```
fd3=open("/etc/testexa", o_RDONLY);
```

```
fd1=open("/etc/test", o_RDWR);
```

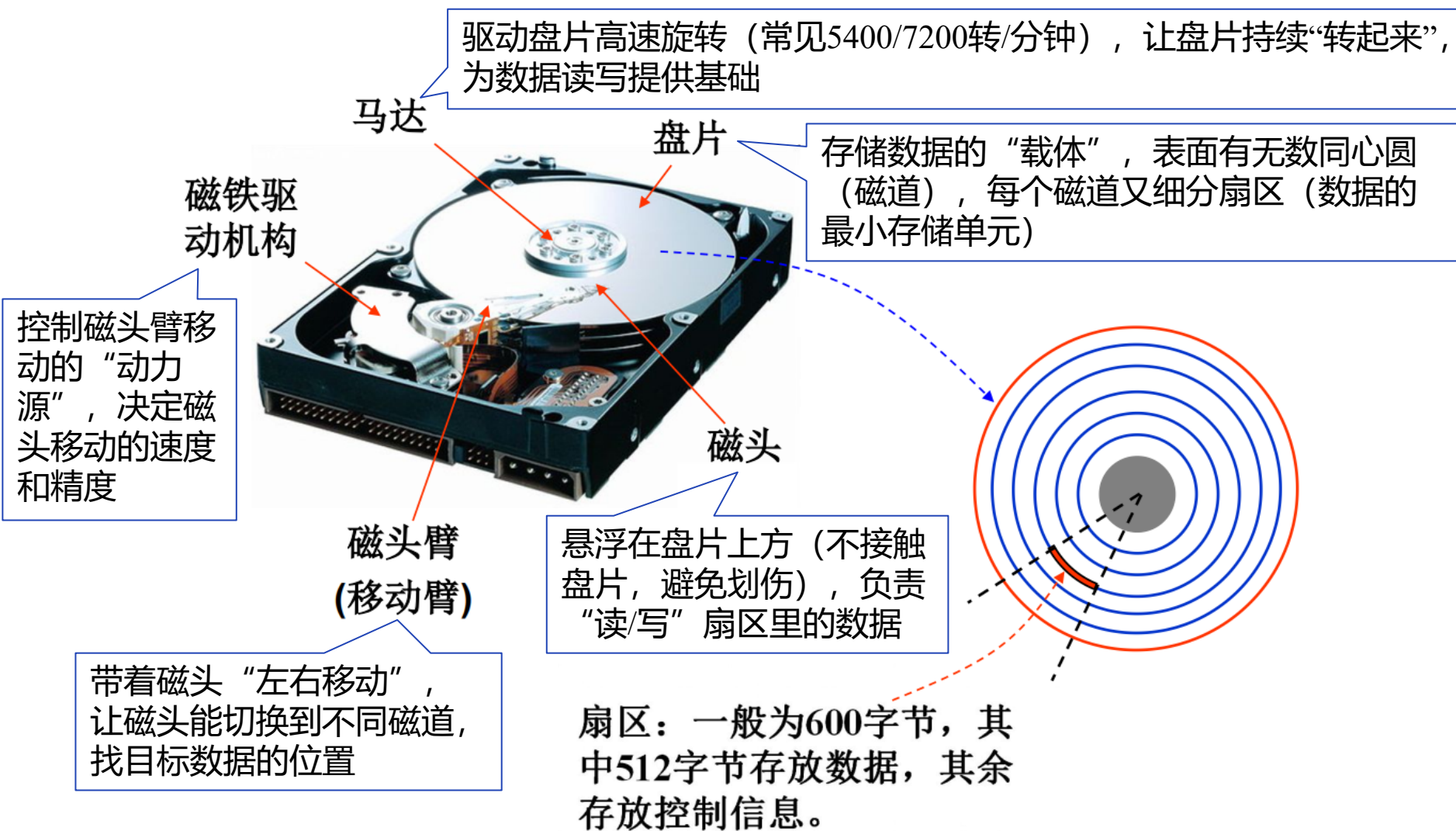


六、文件系统的性能问题

影响文件系统性能的因素？

- 存储介质
- 磁盘性能的好坏
- 磁盘调度算法的好坏
- 磁盘高速缓冲区的大小

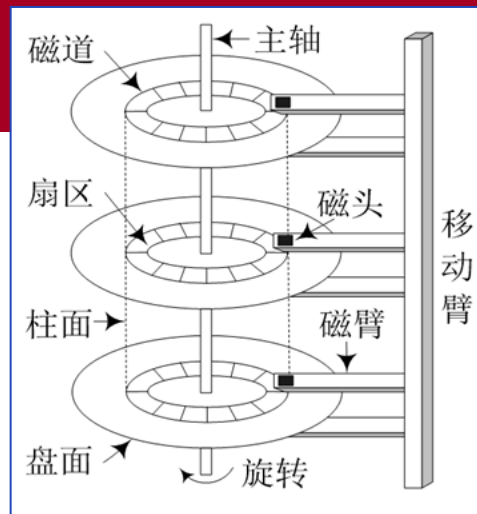
六、文件系统的性能问题



六、文件系统的性能问题

磁盘性能的好坏——典型指标

- **吞吐量**：单位时间能传输的**数据量**（如HDD约100-200MB/s，SSD约500MB/s-7GB/s）；
- **寻道时间（HDD关键）**：**磁头移动到目标磁道的时间（机械硬盘约5-10毫秒）**（机械硬盘存储数据的盘片被划分成了一个一个同心圆，这些同心圆就是**磁道**。磁头悬浮在盘片上方，不与盘片接触，通过移动磁头到不同的磁道来读写数据。当系统下达读取或写入某个数据的指令时，磁头需要从当前位置移动到存储该数据的磁道上，这个移动过程所花费的时间就是**寻道时间**）
- **旋转延迟（HDD关键）**：**磁盘转动到目标扇区的时间（机械硬盘约2-7毫秒）**（盘片在电机的驱动下会以恒定的速度高速旋转，一般常见的转速有5400转/分钟、7200转/分钟等。数据存储在盘片的扇区中，当磁头移动到目标磁道后，还需要等待盘片旋转，使目标扇区转到磁头下方，才能进行数据的读写操作，这段等待盘片旋转的时间就是**旋转延迟**）



六、文件系统的性能问题

寻道时间

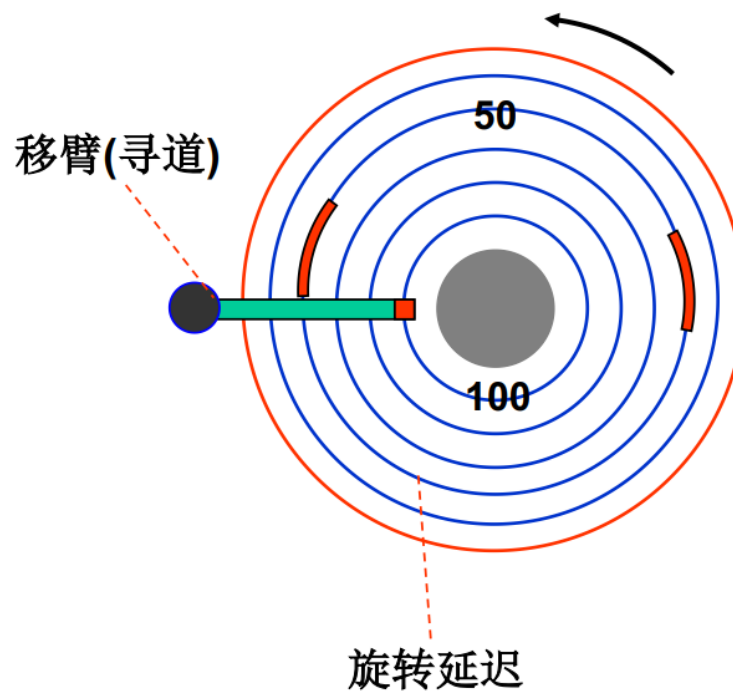
❖ 8-20ms

旋转延迟时间

❖ 4.17ms

数据传输时间

❖ 0.13ms



六、文件系统的性能问题

机械硬盘和固态硬盘的主要区别

特性	机械硬盘	固态硬盘
核心结构	机械部件（盘片、磁头）	闪存芯片 & 主控
工作原理	磁头在旋转盘片上移动读写	纯电子信号读写，无机械运动
速度	慢，受物理运动限制	极快，比机械硬盘快数倍至数十倍
抗震性	差，怕震动、摔落	强，无活动部件，更可靠
噪音	有运行噪音	完全静音
发热与功耗	较高	较低
重量	较重	较轻
价格	容量单价低，性价比高	容量单价高
寿命	理论寿命长，依赖机械磨损	有写入次数限制，但足够日常使用多年
数据恢复	损坏后可能部分恢复	损坏后极难恢复
操作系统影响	需要磁盘调度算法、易产生碎片	无需复杂调度、需 TRIM指令 、碎片影响小

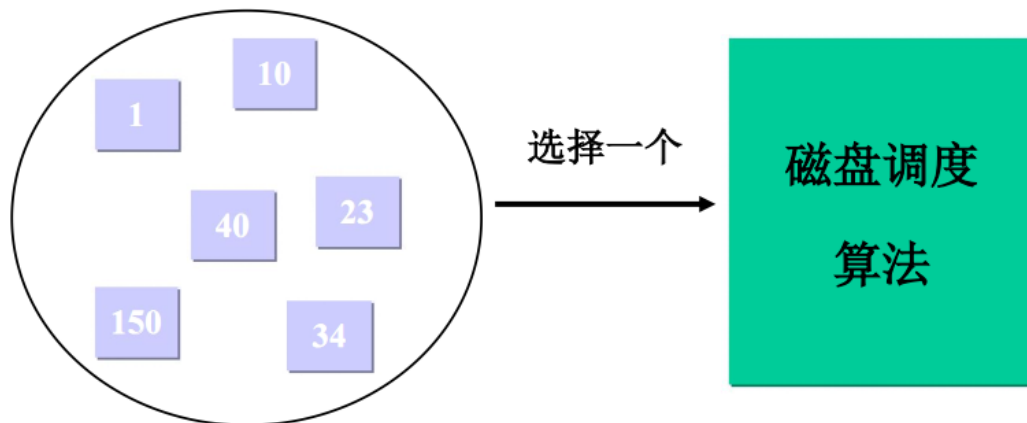
六、文件系统的性能问题

影响文件系统性能的因素？

- 存储介质
- 磁盘性能的好坏
- 磁盘调度算法的好坏
- 磁盘高速缓冲区的大小

磁盘的特点

- ❖ 磁盘是一种共享设备。
- ❖ 不管系统有多少磁头，多少磁道，多少盘片，在任何时候只能有一个磁头处于活动状态。



六、文件系统的性能问题

磁盘调度

- ❖ 磁盘是一种共享设备，为了保证信息的安全，系统每一时刻只允许一个进程启动磁盘进行I/O操作，其余的进程只能等待。因此合理的设置调度算法就十分重要。
- ❖ 设置调度算法的目标：使各进程对磁盘的平均访问时间最小。

□ 固态硬盘——并行能力极强

数据存储在许多闪存芯片（Chip）上，每个芯片内部又分为多个Die（构成闪存芯片最核心的、独立的硅晶片单元），每个Die可以独立操作。

- **多通道**：主控制器与闪存芯片之间通过多条并行通道连接，就像多车道的高速公路。
- **芯片级并行**：不同通道上的芯片可以同时工作。
- **Die级并行**：同一芯片内的不同Die也可以同时工作。
- **Plane级并行**：同一个Die内的不同Plane（可理解为更小的操作单元）也能并行处理操作。

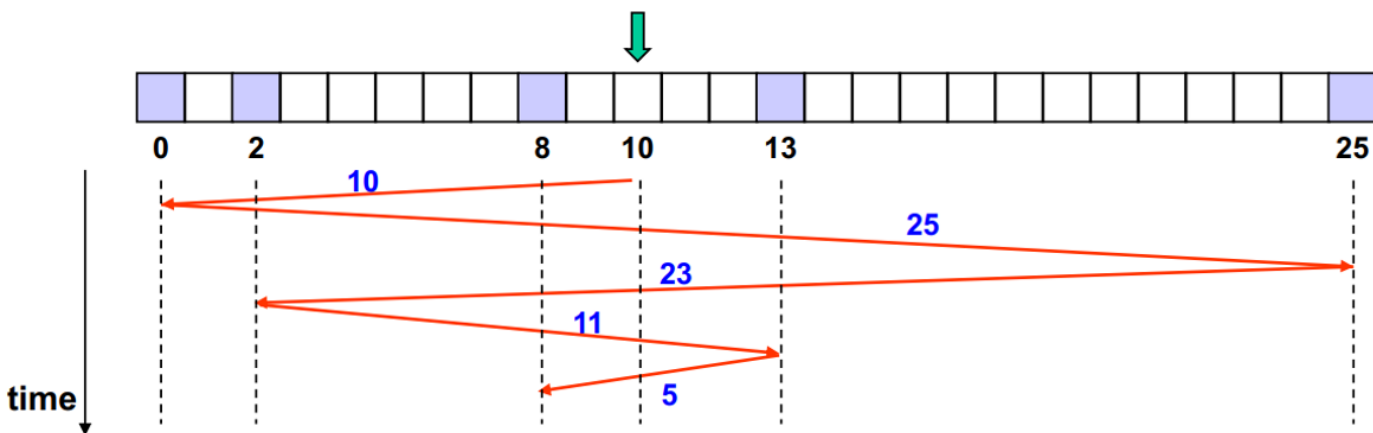
六、文件系统的性能问题



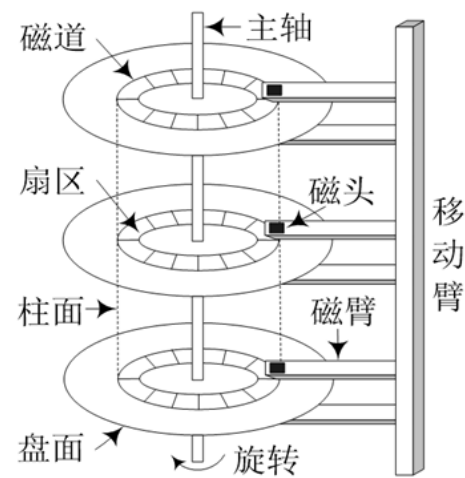
先来先服务算法(FCFS, First Come First Served)

- ❖ 思想：严格按照进程请求访问磁盘的先后次序进行调度，是一种最简单的磁盘调度算法。

磁盘请求顺序：0、25、2、13、8，当前位置：10号柱面



总寻道时间=10+25+23+11+5=74



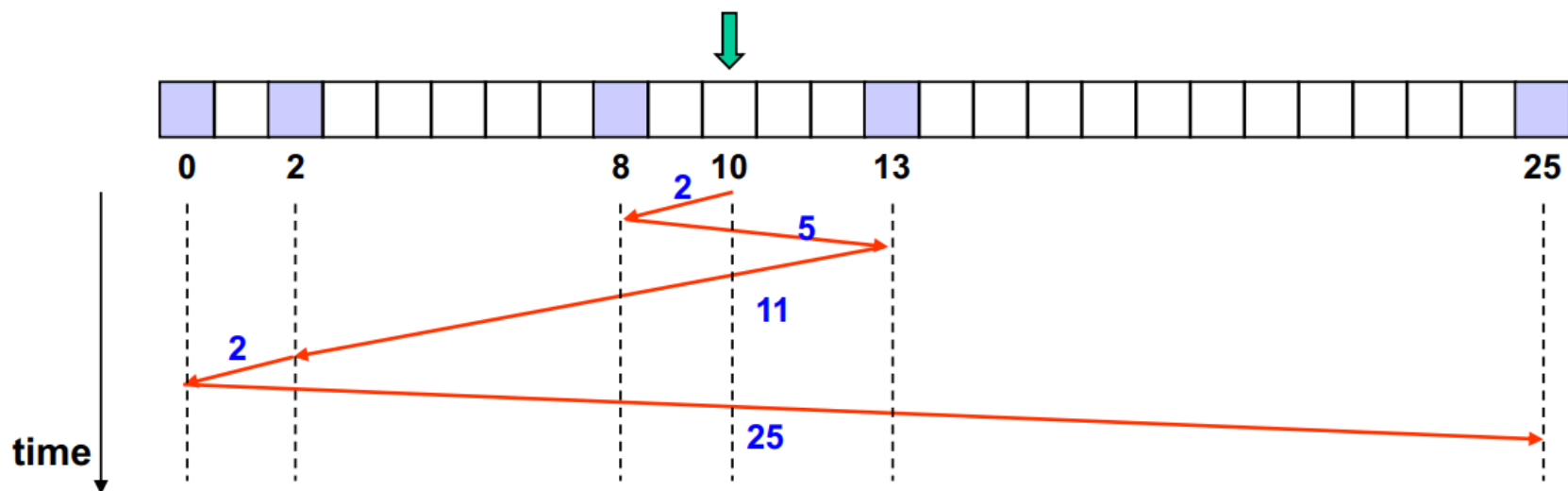
柱面是由机械硬盘中，所有盘片的同一半径位置的磁道所组成的一个逻辑圆柱形结构。

- **优点：**逻辑简单，公平性强（先请求先处理）
- **缺点：**磁头移动距离大，总寻道时间长（比如从0到25再回2，绕路严重）

六、文件系统的性能问题

- ✚ 最短寻道时间优先算法(SSF, Shortest Seek First)
 - ❖ 思想：要求每次访问的磁道与当前磁头所在的磁道距离最近。

磁盘请求顺序：0、25、2、13、8，当前位置：10号柱面



总寻道时间=2+5+11+2+25=45

优点：移臂次数较FCFS算法少。
缺点：可能会出现“饥饿”现象。

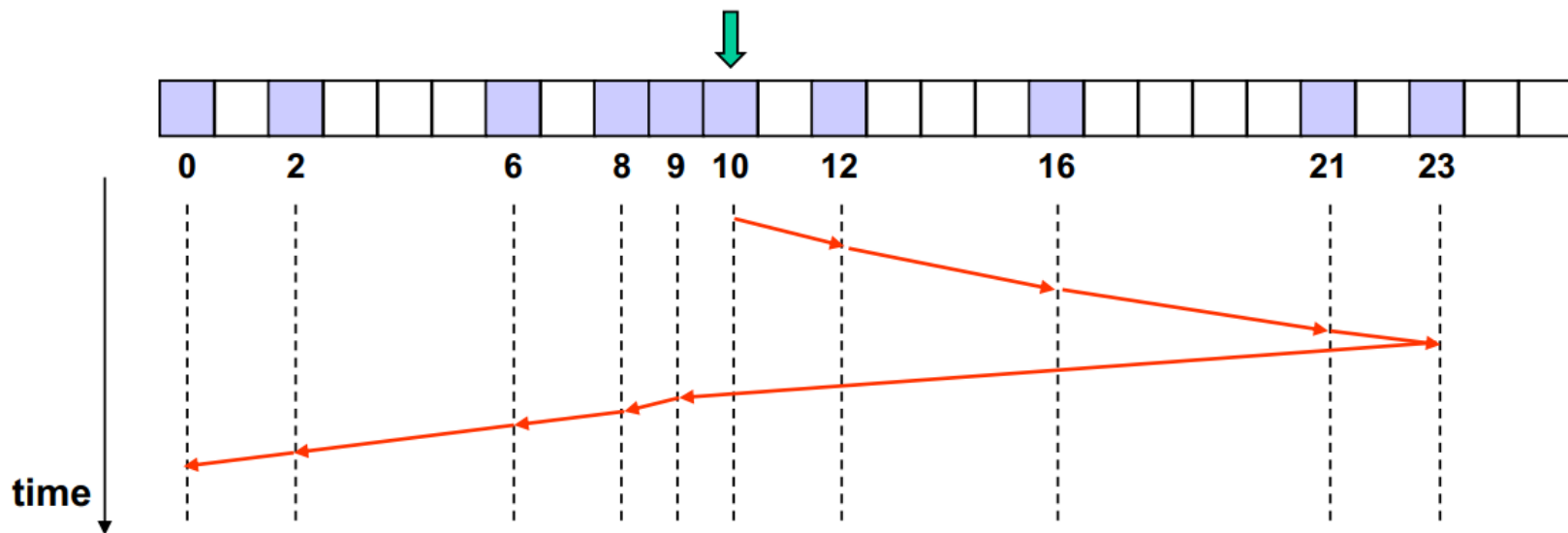
六、文件系统的性能问题



电梯调度算法(SCAN)

- ❖ 思想：不仅考虑到预访问的磁道与当前磁道的距离，更优先考虑磁头的当前移动方向。

磁盘请求顺序：0、23、6、16、2、21、8、9、12，当前位置：10号柱面



总寻道时间=2+4+2+1+2+4+5+2+14=36

优点：可以克服“饥饿”现象，同时可以获得较短的移臂时间。

六、文件系统的性能问题

影响文件系统性能的因素？

- 存储介质
- 磁盘性能的好坏
- 磁盘调度算法的好坏
- 磁盘高速缓冲区的大小

六、文件系统的性能问题

磁盘高速缓存

- **磁盘高速缓存** (Disk Cache) 是指利用内存中的存储空间，来暂存从磁盘中读出的一系列盘块中的信息。
- **磁盘高速缓存的形式：**
 - 专用缓存：**在内存中开辟一个单独的存储空间来作为磁盘高速缓存，大小固定
 - 优点：**分配明确，不会与其他内存需求冲突，缓存策略简单可控
 - 缺点：**空间固定，无法动态调整；若缓存空间过小，无法覆盖高频数据；若过大，浪费内存资源
 - 公用缓存：**把所有未利用的内存空间作为一个缓冲池，供请求分页系统和磁盘I/O共享
 - 优点：**空闲时扩大缓存、内存紧张时收缩，提升整体内存利用率
 - 缺点：**缓存策略复杂（需平衡磁盘I/O和虚拟内存的需求），可能因内存竞争导致双方性能波动

六、文件系统的性能问题

优化数据的分布

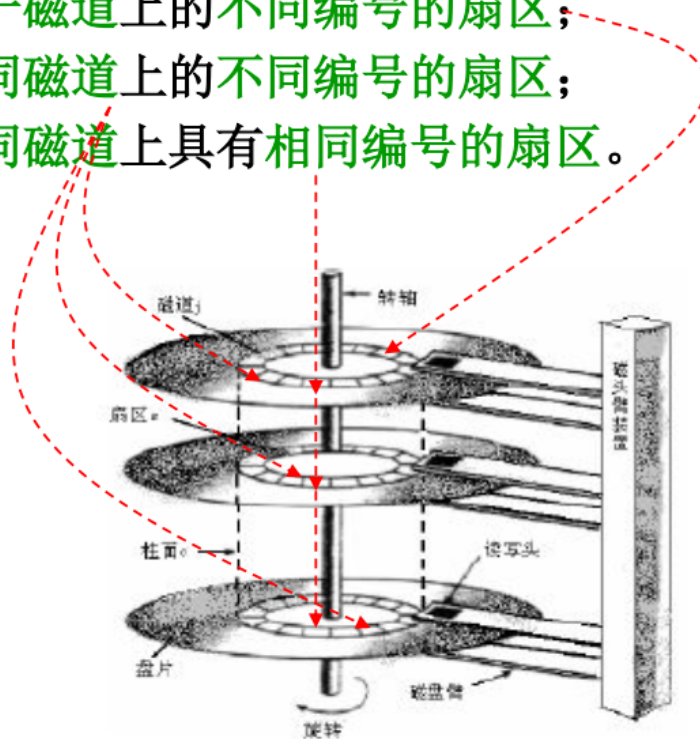
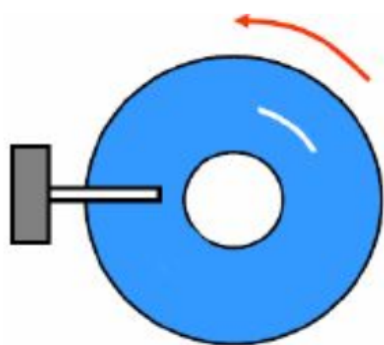
- 优化文件物理块的分布：使磁头的移动距离最少
- 优化索引结点的分布：也会使每次访问文件时，磁头的移动距离减少。

六、文件系统的性能问题

✚ 旋转调度算法

❖ 研究当移动臂定位后，如何访问数据的问题？可能出现的情况如下：

- 进程请求访问的是同一磁道上的不同编号的扇区；
- 进程请求访问的是不同磁道上的不同编号的扇区；
- 进程请求访问的是不同磁道上具有相同编号的扇区。

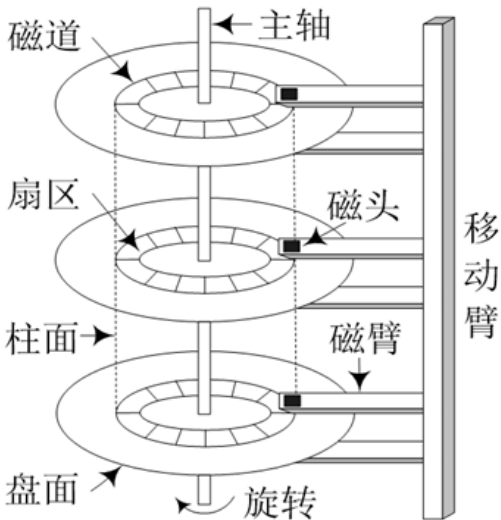


目标：在同一柱面读写信息时，花费最少的时间。

六、文件系统的性能问题

❖ 举例（假设当读写头总是从编号最小的扇区处开始读数据）

请求顺序	柱面号	磁道号	扇区号
①	16	5	3
②	16	1	6
③	16	5	1
④	16	9	6
⑤	16	1	4
⑥	16	8	12



场景1：同一磁道，不同扇区（如柱面16，磁道5，扇区3、1）—— **策略**：按“扇区与磁头当前位置的距离”排序，优先访问“旋转到磁头下时间最短”的扇区。

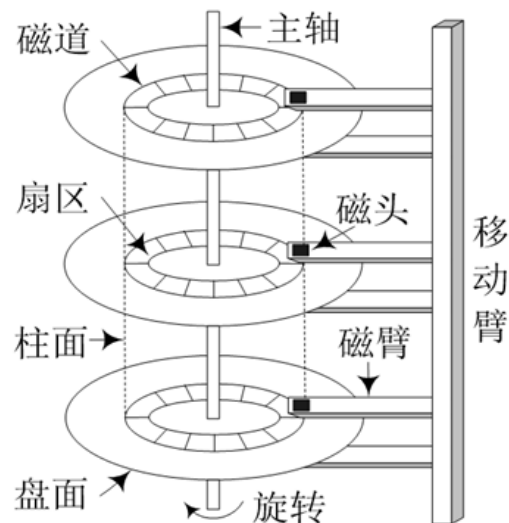
场景2：不同磁道，不同扇区（如柱面16，磁道5扇区3；磁道1扇区6）—— **策略**：排序这些不同磁道（盘面）和不同扇区的请求，优先处理与当前磁头位置相同的请求，然后按扇区旋转到来的顺序，依次切换磁头进行处理，以最小化旋转等待时间。

场景3：不同磁道，相同扇区（如柱面16，磁道5扇区6；磁道9扇区6）—— **策略**：若扇区编号相同，优先访问“磁头移动距离短”的磁道，如磁道5比磁道9近，先访问磁道5的扇区6（磁道不同，移臂不可避免，但旋转调度可优化“同一扇区编号的访问顺序”）。

六、文件系统的性能问题

❖ 举例（假设当读写头总是从编号最小的扇区处开始读数据）

请求顺序	柱面号	磁道号	扇区号
①	16	5	3
②	16	1	6
③	16	5	1
④	16	9	6
⑤	16	1	4
⑥	16	8	12



可能的访问顺序：③①⑤②⑥④，③①⑤④⑥②

旋转延迟的优化策略：磁头到达目标柱面后，不是立即读取当前下方的数据，而是等待并优先读取接下来会经过磁头的、扇区号最小的那个请求。

结论：当一次移臂调度将移动臂定位到某一柱面后，还可能要进行多次旋转调度才能得到所有数据

六、文件系统的性能问题

信息的优化分布

磁盘调度算法可以尽量减少访问时间，但是受到文件存储位置的限制其性能不一定能达到最优。要想进一步提高读写效率，还需要**合理的安排数据的存储位置**。

七、文件系统的可靠性

File system reliability 文件系统的可靠性

- 包含两个方面：
 - **持久性 (Durability)**：数据能长期稳定存储，不因软硬件故障、掉电等丢失
 - **一致性 (Consistency)**：文件系统元数据（如目录、inode）和数据始终符合逻辑规则，避免“**魅影文件**”（无对应元数据的文件）、“**孤儿文件**”（无有效指向的文件）等异常

七、文件系统的可靠性

Backup and Restore 备份和恢复

- **备份:** Use system programs to **back up** data from disk to another storage device (floppy disk, magnetic tape) (用系统程序来**备份**数据到其他的存储设备, 软盘, 磁带) **确保数据永远不会丢失**
- **恢复:** Recover lost file or disk by **restoring** data from backup. (通过从备份重建来**恢复**丢失的文件或磁盘) **还原丢失内容**

七、文件系统的可靠性

Backup

- **备份：**通过转储操作，形成文件或文件系统的多个副本
 - **海量转储：**定期将所有文件拷贝到后援存储器
 - **增量转储：**只转储修改过的文件，即两次备份之间的修改，减少系统开销

七、文件系统的可靠性

文件系统一致性检查

- 文件系统**一致性**包括：
 - 没有“魅影”文件（无对应元数据的文件）
 - 没有“孤儿”文件（无有效指向的文件）
 - 没有消失的空间
 - 没有额外的空间
- 对于后两点，需维护**两张表**：
 - 闲置空间表（记录未被占用的磁盘空间）
 - 占用空间表（记录已被文件占用的空间）
 - 通过比对两张表来确定是否存在消失或额外的空间

七、文件系统的可靠性

- **日志 Logging**

- 将文件操作（如创建、删除、修改）全部记录下来，存放在日志中，用于故障时回滚或恢复

- **事务 Transaction**

- 一个事务是一组操作，事务具有原子性（要么全成功，要么全回滚）

事务的实现需要硬件支持：磁盘的制造商保证一个扇区的读或写是**原子操作**，即该扇区要么全部读/写完，要么根本就没有进行读/写。

七、文件系统的可靠性

磁盘容错技术

- **增加冗余硬件**（磁盘驱动器、磁盘控制器等），来提高磁盘系统的可靠性
 - **低级容错技术**：防止磁盘表面缺陷造成的数据丢失。实现：双份目录双份文件分配表、写后读校验等。
 - **二级容错技术**：防止磁盘驱动器和磁盘控制器的故障所造成的数据丢失。实现：磁盘镜像（Disk Mirroring，双盘同步写）、磁盘双工（Disk Duplexing，双控制器+双盘）。

八、文件系统的安全性

Protection

- 在现代计算机系统中，存放了愈来愈多的宝贵信息供用户使用，给人们带来了极大的好处和方便。
- **影响文件安全性的主要因素**有：
 - **人为因素**：误操作、恶意攻击（如删除文件、篡改数据）；
 - **系统因素**：操作系统漏洞、软硬件故障（如内存错误导致文件损坏）；
 - **自然因素**：火灾、水灾、地震等物理环境破坏。

八、文件系统的安全性

Protection

- 可采取以下**保护措施**：
 - **针对人为因素**：存取控制机制 (Access Control) (如权限管理、用户认证)
 - **防止系统故障**：系统容错技术 (System Fault Tolerance) (如前面的磁盘镜像、日志)
 - **针对自然因素**：后备系统 (Backup System) (如异地备份、灾备中心)

八、文件系统的安全性

File Access Control 文件访问控制 ——存取控制机制

- **文件所有者/创建者的控制能力**：“哪些操作” (**what**) 可被 “谁” (**by whom**) 执行
- **访问类型 (Types of Access)** :
 - Read** (读) : 查看文件内容;
 - Write** (写) : 修改文件内容;
 - Execute** (执行) : 运行可执行文件 (如程序) ;
 - Append** (追加) : 在文件末尾添加内容;
 - Delete** (删除) : 移除文件;
 - List** (列出) : 查看文件元数据 (如名称、大小)

八、文件系统的安全性

File Access Control 文件访问控制

文件的**两级存取控制** (Two-Level File Access Control)

第一级：对访问者的识别

- 对用户分类：
 - 文件主 (**owner**)：创建文件的用户，对文件有最高控制权
 - 文件主的同组用户 (**group**)：与文件主同属一个用户组的其他用户，权限通常低于文件主
 - 其它用户 (**other**)：既非文件主、也非同组的用户，权限最严格

八、文件系统的安全性

File Access Control 文件访问控制

第二级：对操作权限的识别

- 对操作分类：
 - 读操作 (**r**, **read**) : 查看文件内容;
 - 写操作 (**w**, **write**) : 修改文件内容 (包括创建、删除、编辑)
 - 执行操作 (**x**, **execute**) : 运行可执行文件 (如程序、脚本) ;
 - 不能执行任何操作 (**-**) : 拒绝访问文件

八、文件系统的安全性

Hierarchical Security Management 分级安全管理

- 为保证文件的安全性，通常采用**分级的管理措施**：
 - **系统级管理** (System-Level Management)：控制用户能否进入系统
 - **用户级管理** (User-Level Management)：给用户分配文件访问权
 - **目录级管理** (Directory-Level Management)：保护目录结构安全
 - **文件级管理** (File-Level Management)：直接控制文件的访问权限

八、文件系统的安全性

系统级安全管理

- 其主要任务是**不允许未经核准的用户进入系统**
- 系统级安全管理：
 - **注册** (Registration) : 便于系统管理员对用户进行管理, 并保证用户名在系统中的唯一性
 - **登录** (Login) : 通过核实该用户的注册名及口令来检查该用户使用系统的合法性

八、文件系统的安全性

用户验证

- **当用户登录时，检验其身份**
(用户是谁，用户拥有什么，用户知道什么)
 - 口令
 - 物理鉴定
(磁卡，指纹，签名分析，手指长度分析)

八、文件系统的安全性

用户级安全管理

- 用户级安全管理，是**为了给用户分配“文件访问权”**而设计的，包括：
 - **对所有用户进行分类**：区分文件主、同组用户、其他用户等
 - **为指定用户分配文件访问权**：为不同分类的用户设置读、写、执行等权限
 - **采用保护域**：限定用户能访问的资源范围，避免越权

八、文件系统的安全性

目录级安全管理

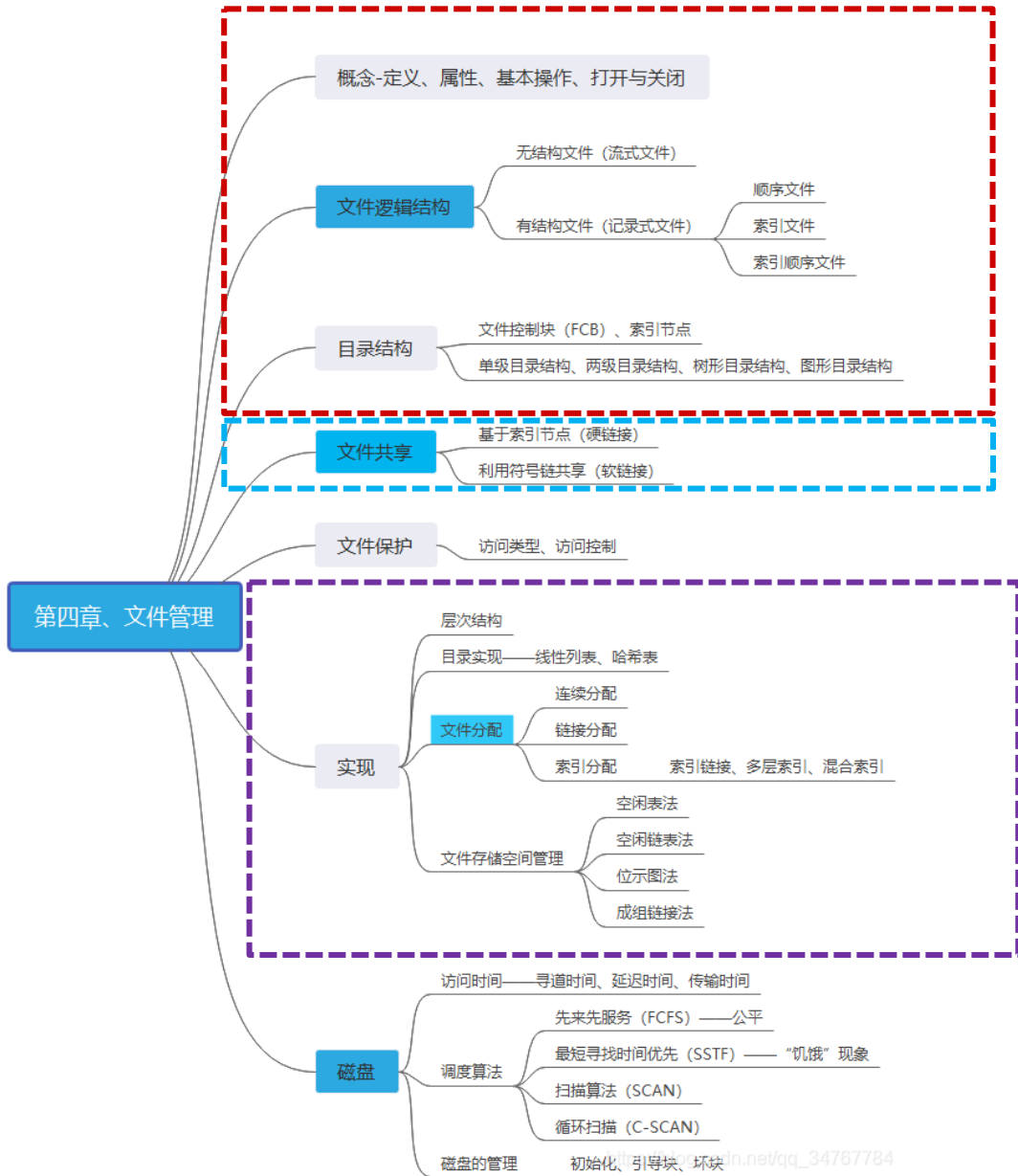
- 目录级安全管理，是**为保护系统中的各种目录**而设计的，与用户权限无关
- 为保证目录的安全，规定只有**系统核心**才具有写目录（修改目录结构）的权利

八、文件系统的安全性

文件级安全管理

- 通过系统管理员或文件主对文件属性的设置，来**控制用户对文件访问**
- 采用**文件访问控制**（如UNIX等系统采用**存取控制表**，记录不同用户/用户组对文件的操作权限）

小结



论述（思考）：文件系统的软硬件协调问题